# OS_homework_3

这份文件是OS_homework_3 by计科210X 甘晴void 202108010XXX

文档设置了目录，可以通过目录快速跳转至答案部分。

## 第26章

运行该程序wolf/OS-homework/threads-intro/x86.py

通过README可知一些重要的标识符如下

```
-p：指定程序
-t：线程数
-i：指定每执行i个指令产生一次中断
-M：追踪内存
-R：追踪特定寄存器值
-C：追踪条件码的值
-r：中断随机
```

## 26.1

> 开始,我们来看一个简单的程序,*“loop.s”*。首先,阅读这个程序,看看你是否能理解它:*cat loop.s*。然后,用这些参数运行它:`./x86.py -p loop.s -t 1 -i 100 -R dx`
>
> 这指定了一个单线程,每 *100* 条指令产生一个中断,并且追踪寄存器 *%d*。你能弄清楚 *%dx* 在运行过程中的值吗*?* 你有答案之后,运行上面的代码并使用 *-c* 标志来检查你的答案。注意答案的左边显示了右侧指令运行后寄存器的值*(*或内存的值*)*

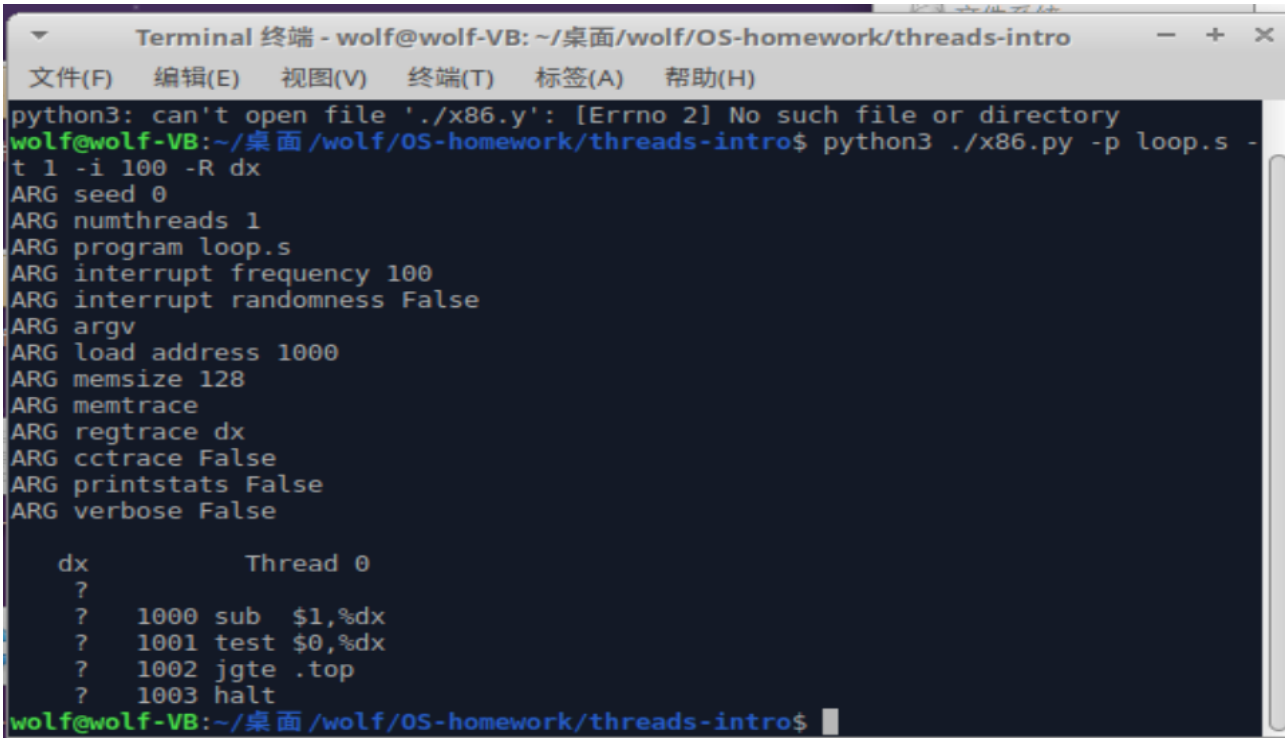
使用cat命令查看loop.s文件

```
.main
.top            // 标号
sub  $1,%dx     // dx寄存器值减1, 结果存入dx
test $0,%dx     // 比较 dx寄存器的值与 0
jgte .top       // 如果 dx 寄存器的值大于或等于 0,则跳转到标号 .top 处
halt            // 结束线程
```

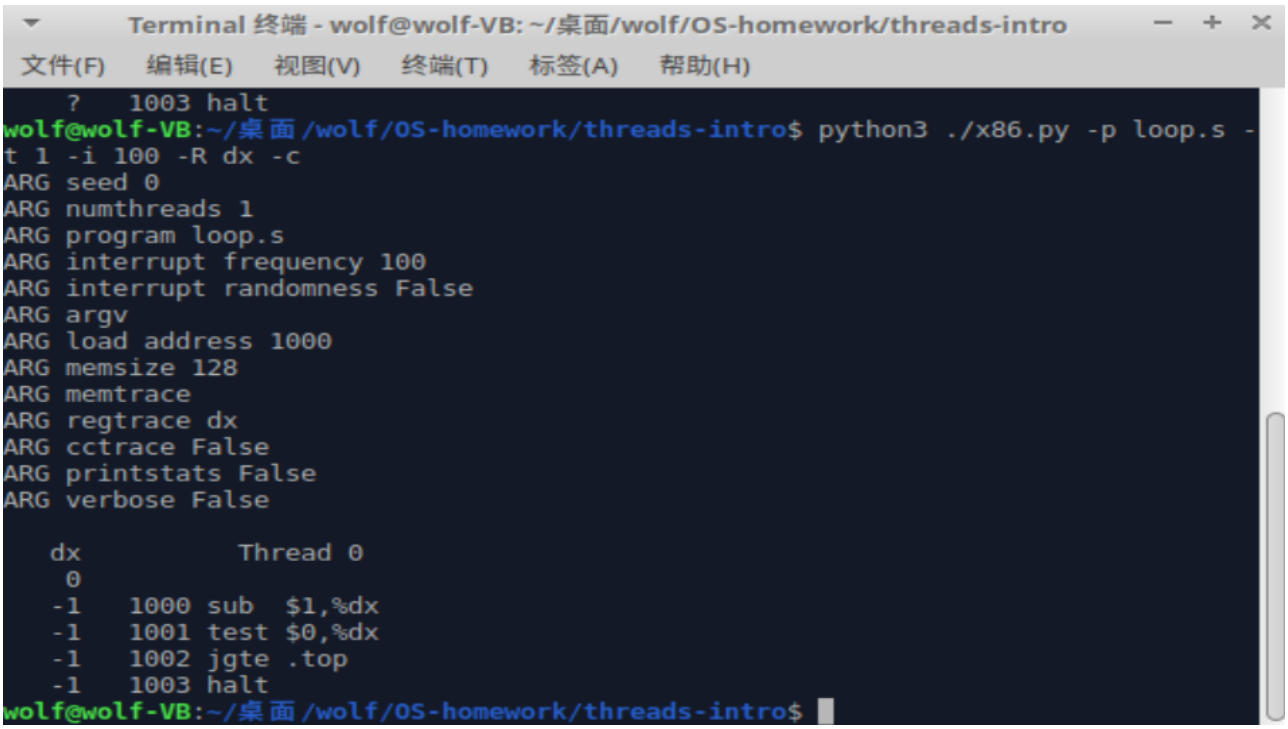
这段程序实现了一个简单循环的单线程，，当%dx内的值小于0时跳出循环。

下面运行该程序

```
./x86.py -p loop.s -t 1 -i 100 -R dx
```



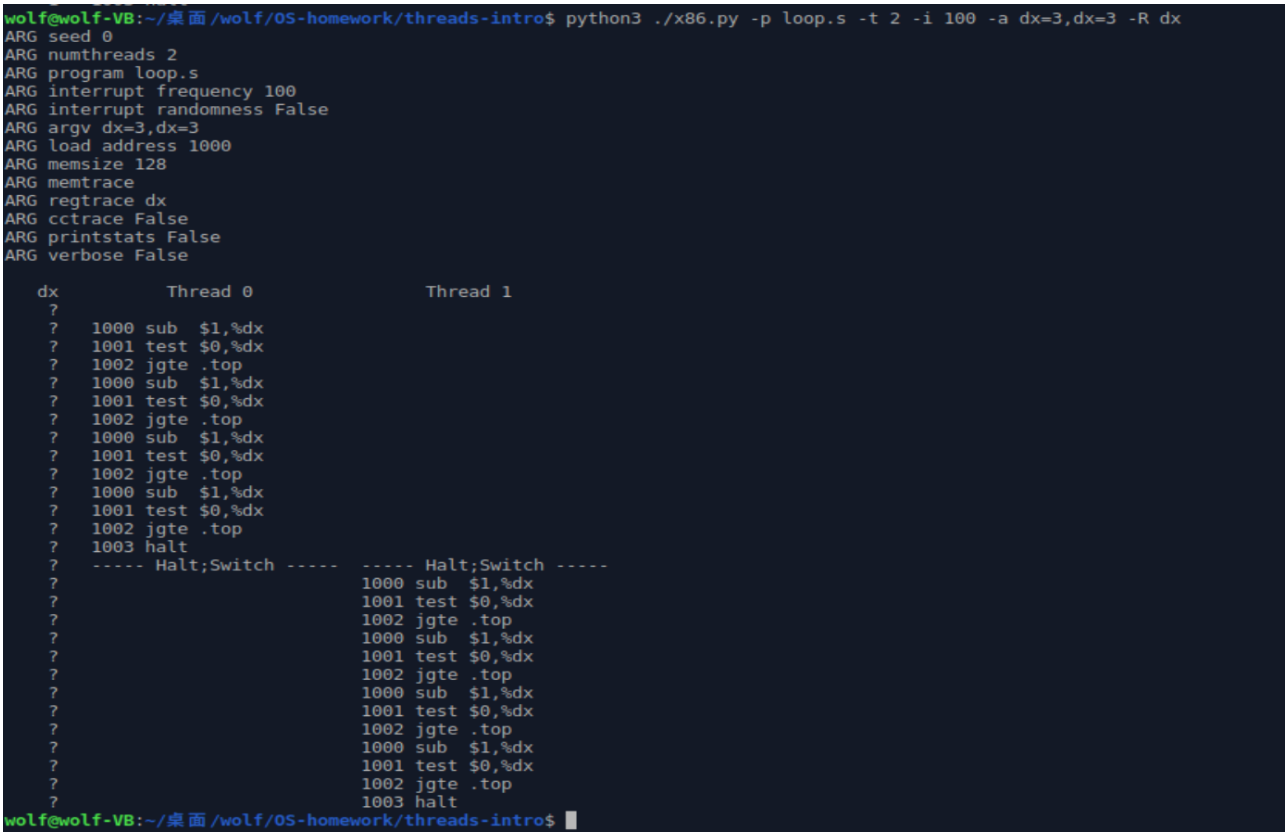

只执行了4条指令就结束了，dx的初始值应该为0，进入循环后减为-1，直接退出了循环。使用-c标志检查，dx的值符合预期。



## 26.2

> 现在运行相同的代码,但使用这些标志:
>
> `./x86.py -p loop.s -t 2 -i 100 -a dx=3,dx=3 -R dx`
>
> 这指定了两个线程,并将每个*%dx* 寄存器初始化为 *3*. *%dx* 会看到什么值*?*使用*-c* 标志运行以查看答案。多个线程的存在是否会影响计算？这段代码有竞态条件吗？

使用如上标志运行代码，运行截图如下




使用-c标志查看%dx的值

```
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-intro$ python3 ./x86.py -p loop.s -t 2 -i 100 -a dx=3,dx=3 -R dx -c
ARG seed 0
ARG numthreads 2
ARG program loop.s
ARG interrupt frequency 100
ARG interrupt randomness False
ARG argv dx=3,dx=3
ARG load address 1000
ARG memsize 128
ARG memtrace
ARG regtrace dx
ARG cctrace False
ARG printstats False
ARG verbose False

   dx          Thread 0                Thread 1
    3
    2   1000 sub  $1,%dx
    2   1001 test $0,%dx
    2   1002 jgte .top
    1   1000 sub  $1,%dx
    1   1001 test $0,%dx
    1   1002 jgte .top
    0   1000 sub  $1,%dx
    0   1001 test $0,%dx
    0   1002 jgte .top
   -1   1000 sub  $1,%dx
   -1   1001 test $0,%dx
   -1   1002 jgte .top
   -1   1003 halt
    3   ----- Halt;Switch -----  ----- Halt;Switch -----
    2                           1000 sub  $1,%dx
    2                           1001 test $0,%dx
    2                           1002 jgte .top
    1                           1000 sub  $1,%dx
    1                           1001 test $0,%dx
    1                           1002 jgte .top
    0                           1000 sub  $1,%dx
    0                           1001 test $0,%dx
    0                           1002 jgte .top
   -1                           1000 sub  $1,%dx
   -1                           1001 test $0,%dx
   -1                           1002 jgte .top
   -1                           1003 halt
```

线程 1 运行结果与线程 0 相同，dx的值最后都为-1。 多个线程不会影响计算,因为指令执行时长小于中断周期。

这段代码没有竞态条件。因为并没有出现两个线程同时进入临界区的情况。

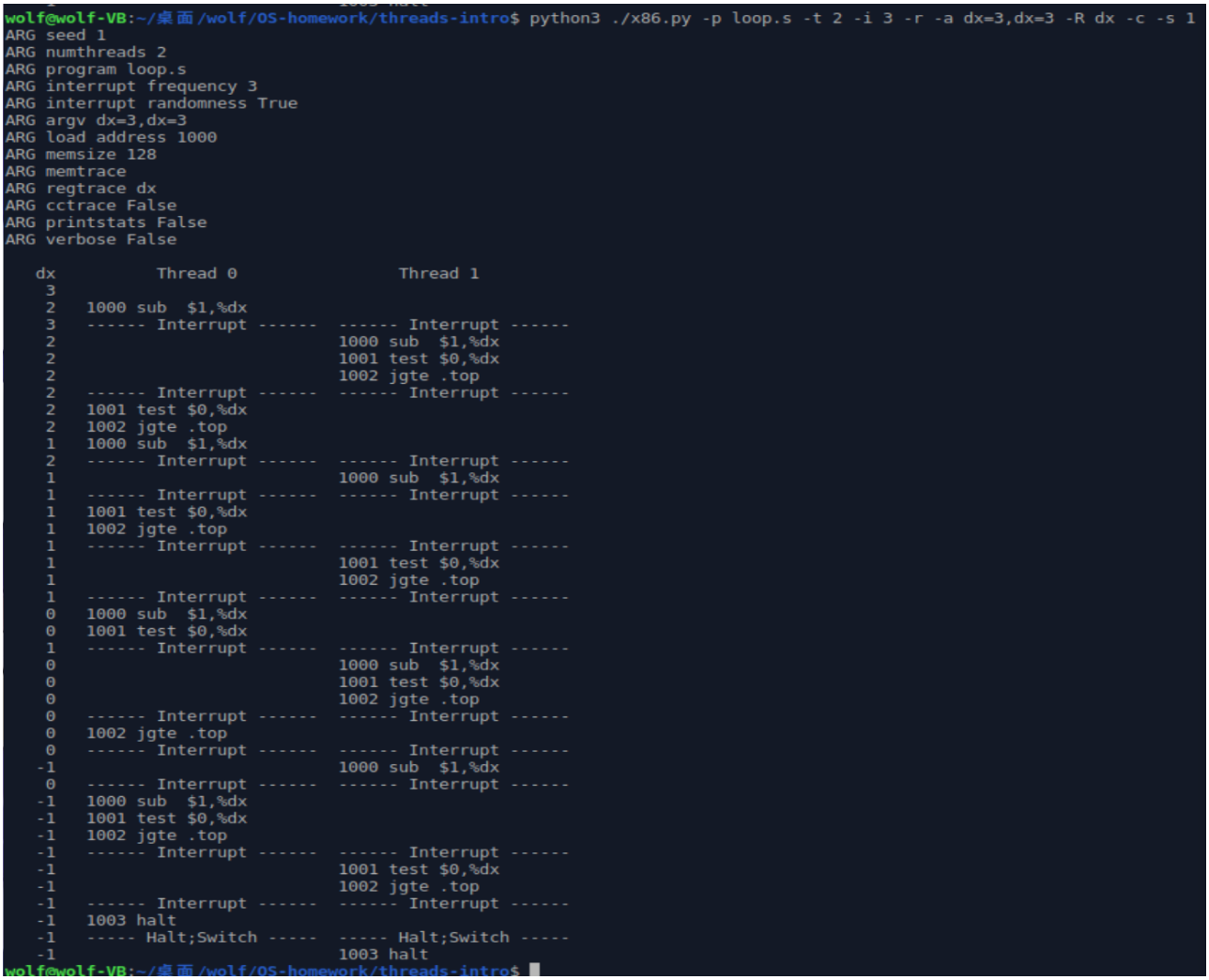
## 26.3

> 现在运行以下命令:
>
> `./x86.py -p loop.s -t 2 -i 3 -r -a dx=3,dx=3 -R dx`
>
> 这使得中断间隔非常小且随机，使用不同的种子和*-s* 来查看不同的交替、中断频率是否会改变这个程序的行为?

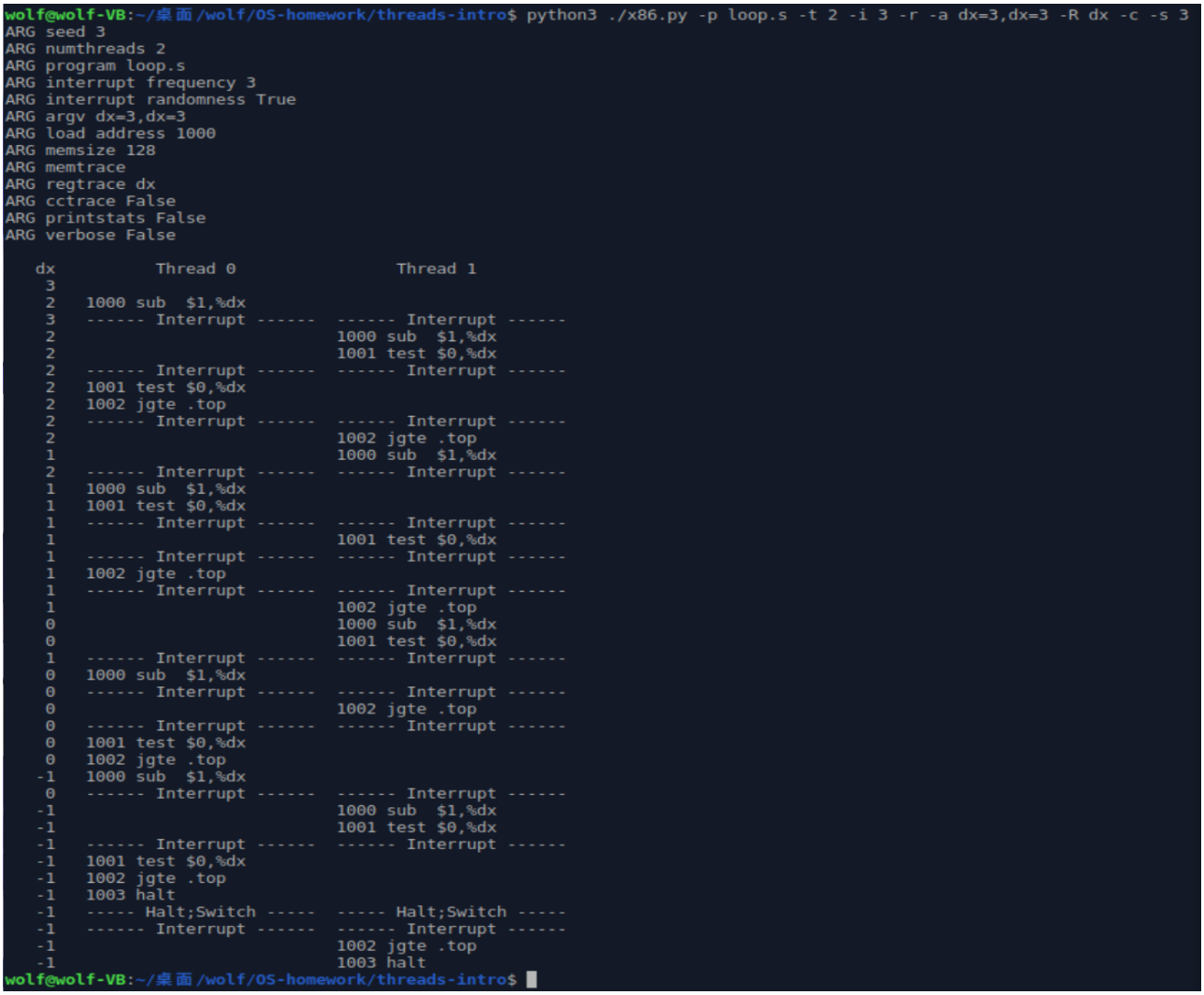
分别执行如下命令

```
./x86.py -p loop.s -t 2 -i 3 -r -a dx=3,dx=3 -R dx -c -s 1
./x86.py -p loop.s -t 2 -i 3 -r -a dx=3,dx=3 -R dx -c -s 3
./x86.py -p loop.s -t 2 -i 3 -r -a dx=3,dx=3 -R dx -c -s 5
```

种子为1时的运行截图


wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-intro$ python3 ./x86.py -p loop.s -t 2 -i 3 -r -a dx=3,dx=3 -R dx -c -s 1
ARG seed 1
ARG numthreads 2
ARG program loop.s
ARG interrupt frequency 3
ARG interrupt randomness True
ARG argv dx=3,dx=3
ARG load address 1000
ARG memsize 128
ARG memtrace
ARG regtrace dx
ARG cctrace False
ARG printstats False
ARG verbose False
dx Thread 0 Thread 1
3
2 1000 sub $1,%dx
3 ------ Interrupt ------ ------ Interrupt ------
2 1000 sub $1,%dx
2 1001 test $0,%dx
2 1002 jgte .top
2 ------ Interrupt ------ ------ Interrupt ------
2 1001 test $0,%dx
2 1002 jgte .top
1 1000 sub $1,%dx
2 ------ Interrupt ------ ------ Interrupt ------
1 1000 sub $1,%dx
1 ------ Interrupt ------ ------ Interrupt ------
1 1001 test $0,%dx
1 1002 jgte .top
1 ------ Interrupt ------ ------ Interrupt ------
1 1001 test $0,%dx
1 1002 jgte .top
1 ------ Interrupt ------ ------ Interrupt ------
0 1000 sub $1,%dx
0 1001 test $0,%dx
1 ------ Interrupt ------ ------ Interrupt ------
0 1000 sub $1,%dx
0 1001 test $0,%dx
0 1002 jgte .top
0 ------ Interrupt ------ ------ Interrupt ------
0 1002 jgte .top
0 ------ Interrupt ------ ------ Interrupt ------
-1 1000 sub $1,%dx
0 ------ Interrupt ------ ------ Interrupt ------
-1 1000 sub $1,%dx
-1 1001 test $0,%dx
-1 1002 jgte .top
-1 ------ Interrupt ------ ------ Interrupt ------
-1 1001 test $0,%dx
-1 1002 jgte .top
-1 ------ Interrupt ------ ------ Interrupt ------
-1 1003 halt
-1 ----- Halt;Switch ----- ----- Halt;Switch -----
-1 1003 halt
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-intro$

种子为3时的运行截图


wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-intro$ python3 ./x86.py -p loop.s -t 2 -i 3 -r -a dx=3,dx=3 -R dx -c -s 3
ARG seed 3
ARG numthreads 2
ARG program loop.s
ARG interrupt frequency 3
ARG interrupt randomness True
ARG argv dx=3,dx=3
ARG load address 1000
ARG memsize 128
ARG memtrace
ARG regtrace dx
ARG cctrace False
ARG printstats False
ARG verbose False
dx Thread 0 Thread 1
3
2 1000 sub $1,%dx
3 ------ Interrupt ------ ------ Interrupt ------
2 1000 sub $1,%dx
2 1001 test $0,%dx
2 ------ Interrupt ------ ------ Interrupt ------
2 1001 test $0,%dx
2 1002 jgte .top
2 ------ Interrupt ------ ------ Interrupt ------
2 1002 jgte .top
1 1000 sub $1,%dx
2 ------ Interrupt ------ ------ Interrupt ------
1 1000 sub $1,%dx
1 1001 test $0,%dx
1 ------ Interrupt ------ ------ Interrupt ------
1 1001 test $0,%dx
1 ------ Interrupt ------ ------ Interrupt ------
1 1002 jgte .top
1 ------ Interrupt ------ ------ Interrupt ------
1 1002 jgte .top
0 1000 sub $1,%dx
0 1001 test $0,%dx
1 ------ Interrupt ------ ------ Interrupt ------
0 1000 sub $1,%dx
0 ------ Interrupt ------ ------ Interrupt ------
0 1002 jgte .top
0 ------ Interrupt ------ ------ Interrupt ------
0 1001 test $0,%dx
0 1002 jgte .top
-1 1000 sub $1,%dx
0 ------ Interrupt ------ ------ Interrupt ------
-1 1000 sub $1,%dx
-1 1001 test $0,%dx
-1 ------ Interrupt ------ ------ Interrupt ------
-1 1001 test $0,%dx
-1 1002 jgte .top
-1 1003 halt
-1 ----- Halt;Switch ----- ----- Halt;Switch -----
-1 ------ Interrupt ------ ------ Interrupt ------
-1 1002 jgte .top
-1 1003 halt
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-intro$

种子为5时的运行截图

```
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-intro$ python3 ./x86.py -p loop.s -t 2 -i 3 -r -a dx=3,dx=3 -R dx -c -s 5
ARG seed 5
ARG numthreads 2
ARG program loop.s
ARG interrupt frequency 3
ARG interrupt randomness True
ARG argv dx=3,dx=3
ARG load address 1000
ARG memsize 128
ARG memtrace
ARG regtrace dx
ARG cctrace False
ARG printstats False
ARG verbose False

   dx          Thread 0                Thread 1
    3
    2   1000 sub  $1,%dx
    2   1001 test $0,%dx
    3   ------ Interrupt ------  ------ Interrupt ------
    2                           1000 sub  $1,%dx
    2                           1001 test $0,%dx
    2                           1002 jgte .top
    2   ------ Interrupt ------  ------ Interrupt ------
    2   1002 jgte .top
    1   1000 sub  $1,%dx
    1   1001 test $0,%dx
    2   ------ Interrupt ------  ------ Interrupt ------
    1                           1000 sub  $1,%dx
    1                           1001 test $0,%dx
    1                           1002 jgte .top
    1   ------ Interrupt ------  ------ Interrupt ------
    1   1002 jgte .top
    0   1000 sub  $1,%dx
    0   1001 test $0,%dx
    1   ------ Interrupt ------  ------ Interrupt ------
    0                           1000 sub  $1,%dx
    0                           1001 test $0,%dx
    0                           1002 jgte .top
    0   ------ Interrupt ------  ------ Interrupt ------
    0   1002 jgte .top
    0   ------ Interrupt ------  ------ Interrupt ------
   -1                           1000 sub  $1,%dx
   -1                           1001 test $0,%dx
    0   ------ Interrupt ------  ------ Interrupt ------
   -1   1000 sub  $1,%dx
   -1   1001 test $0,%dx
   -1   1002 jgte .top
   -1   ------ Interrupt ------  ------ Interrupt ------
   -1                           1002 jgte .top
   -1                           1003 halt
   -1   ----- Halt;Switch -----  ----- Halt;Switch -----
   -1   ------ Interrupt ------  ------ Interrupt ------
   -1   1003 halt
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-intro$
```

由于这两个子进程并不共享变量，也就是没有临界区。因此，两个子进程交替运行并不会产生竞态条件。故中断频率没有改变这个程序的行为。

## 26.4

> 接下来我们将研究一个不同的程序（*looping-race-nolock.s*）,该程序访问位于内存地址 *2000* 的共享变量，简单起见，我们称这个变量为 *x*.使用单线程运行它，并确保你了解它的功能，如下所示:
>
> `./x86.py -p looping-race-nolock.s -t 1 -M 2000`
>
> 在整个运行过程中， *x*（即内存地址为 *2000*）的值是多少？使用*-c* 来检查你的答案。

使用`cat looping-race-nolock.s`查看该程序如下

```
# assumes %bx has loop count in it

.main
.top
# critical section
```

```
mov 2000, %ax  # get 'value' at address 2000
add $1, %ax    # increment it
mov %ax, 2000  # store it back

# see if we're still looping
sub  $1, %bx
test $0, %bx
jgt .top

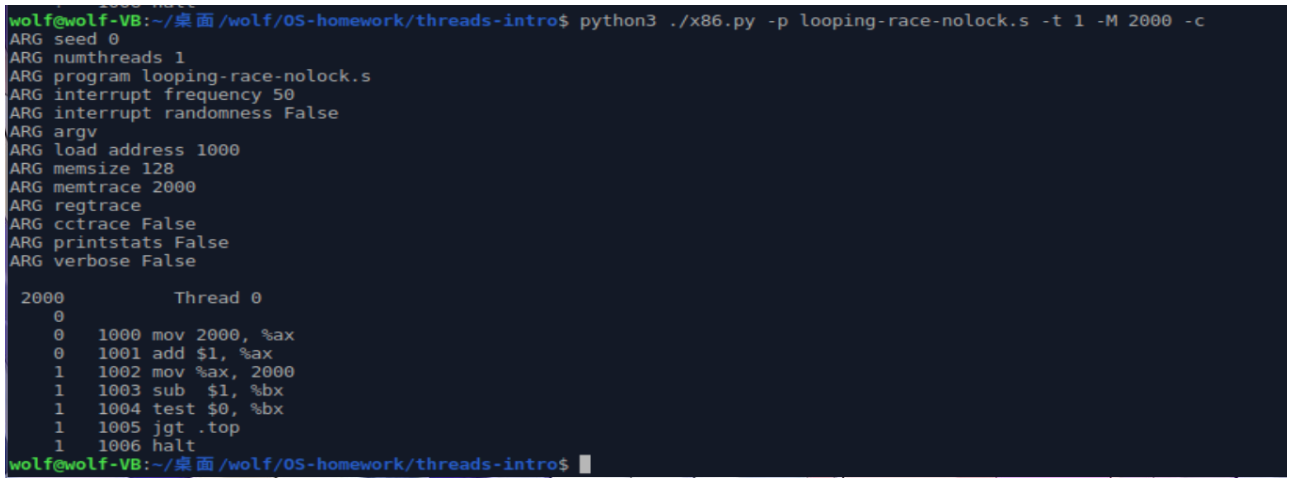
halt
```

这个程序使用循环来对地址2000的变量x进行累加%bx次。

具体来说如下：

- 将内存地址2000中的值赋给寄存器%ax
- 将寄存器%ax中的值加1
- 将寄存器%ax中的值重新写回内存地址2000中
- 根据寄存%bx中的值执行以上循环(这里未设置，默认bx为0，执行一次)

如果仅仅是上述代码的话，运行可得结果是x=1。运行截图如下：




但是这道题的代码是有竞态条件的，x所在的地址这个位置是临界区。

如果两个同样的该线程运行，就会对地址2000（x）处的共享数据进行操作。两个线程运行的正确的效果应该为最终x=原x+2dx的值，但同时进入临界区将导致错误发生。有可能结果为最终x=原x+dx的值。（这里详细的部分书p202-203有详细讲述）

# 第28章

运行程序wolf/OS-homework/threads-locks/x86.py

## 28.1

> 首先用标志 *flag.s*运行 *x86.py*。该代码通过一个内存标志“实现”锁。你能理解汇编代码试图做什么吗?

查看汇编代码如下

```
.var flag
.var count

.main
.top

.acquire
mov  flag, %ax      # get flag
test $0, %ax        # if we get 0 back: lock is free!
jne  .acquire       # if not, try again
mov  $1, flag       # store 1 into flag

# critical section
mov  count, %ax     # get the value at the address
add  $1, %ax        # increment it
mov  %ax, count     # store it back

# release lock
mov  $0, flag       # clear the flag now

# see if we're still looping
sub  $1, %bx
test $0, %bx
jgt .top

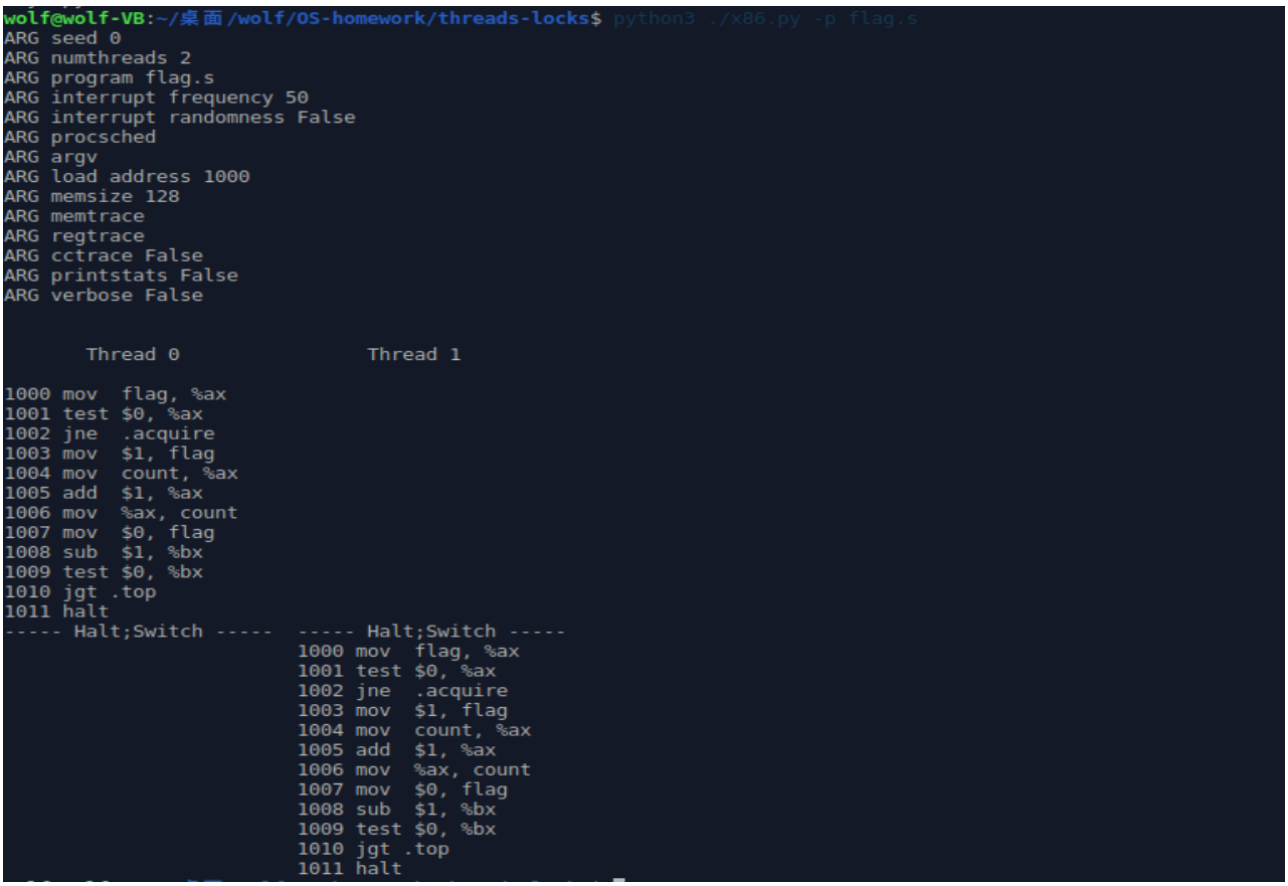
halt
```

很明显，这里试图在flag内储存我们锁的开关状态，对临界区代码进行保护。也就是书上28.1所讲述的简单锁的一个尝试。

执行；如下代码

```
python3 ./x86.py -p flag.s
```

运行截图如下：




但这个实现是有问题的，会出现类似如下的错误：

| 线程1 | 线程2 |
| --- | --- |
| mov flag, %ax<br>test $0, %ax | |
| | mov flag, %ax<br>test $0, %ax<br>mov $1, flag |
| mov $1, flag<br>… | |
| | … |

在不恰当的中断下，有可能会使线程1和线程2的flag都为1，从而都持有锁并进入临界区，导致事实上锁没有起到应该具有的作用。

## 28.2

> 使用默认值运行时, *flag.s*是否按预期工作?它会产生正确的结果吗?使用*-M* 和*-R* 标志跟踪变量和寄存器(并使用*-c* 查看它们的值)。你能预测代码运行时 *flag* 最终会变成什么值吗?

使用默认值运行时, flag.s会按预期工作，并且能产生预期的结果。

这一点在上一题已经验证过了。

执行如下命令

```
./x86.py -p flag.s -R ax,bx -M flag,count -c
```

运行截图如下：

```
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-locks$ python3 ./x86.py -p flag.s -R ax,bx -M flag,count -c
ARG seed 0
ARG numthreads 2
ARG program flag.s
ARG interrupt frequency 50
ARG interrupt randomness False
ARG procsched
ARG argv
ARG load address 1000
ARG memsize 128
ARG memtrace flag,count
ARG regtrace ax,bx
ARG cctrace False
ARG printstats False
ARG verbose False

 flag count      ax    bx          Thread 0                Thread 1

    0     0       0     0
    0     0       0     0   1000 mov  flag, %ax
    0     0       0     0   1001 test $0, %ax
    0     0       0     0   1002 jne  .acquire
    1     0       0     0   1003 mov  $1, flag
    1     0       0     0   1004 mov  count, %ax
    1     0       1     0   1005 add  $1, %ax
    1     1       1     0   1006 mov  %ax, count
    0     1       1     0   1007 mov  $0, flag
    0     1       1    -1   1008 sub  $1, %bx
    0     1       1    -1   1009 test $0, %bx
    0     1       1    -1   1010 jgt .top
    0     1       1    -1   1011 halt
    0     1       0     0   ----- Halt;Switch -----  ----- Halt;Switch -----
    0     1       0     0                           1000 mov  flag, %ax
    0     1       0     0                           1001 test $0, %ax
    0     1       0     0                           1002 jne  .acquire
    1     1       0     0                           1003 mov  $1, flag
    1     1       1     0                           1004 mov  count, %ax
    1     1       2     0                           1005 add  $1, %ax
    1     2       2     0                           1006 mov  %ax, count
    0     2       2     0                           1007 mov  $0, flag
    0     2       2    -1                           1008 sub  $1, %bx
    0     2       2    -1                           1009 test $0, %bx
    0     2       2    -1                           1010 jgt .top
    0     2       2    -1                           1011 halt
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-locks$
```

在默认情况下，中断间隔为50，线程运行没有发生中断，两个线程先后运行，不会产生错误，最终标志flag会在线程2最后一次循环被设置为0，而count=count+2*bx值。

## 28.3

> 使用*-a* 标志更改寄存器*%bx* 的值*(*例如*,*如果只运行两个线程*,*就用*-a bx=2,bx=2)*。代码是做什么的*?*对这段代码问上面的问题*,*答案如何*?*

将bx值设置为2，运行两个线程，则每个线程实现count=count+2，最终count+=4。在题1中已分析过线程执行的代码存在竞态条件，但由于50条指令才发生中断，此处两个线程仍然是先后运行的，不会产生错误。

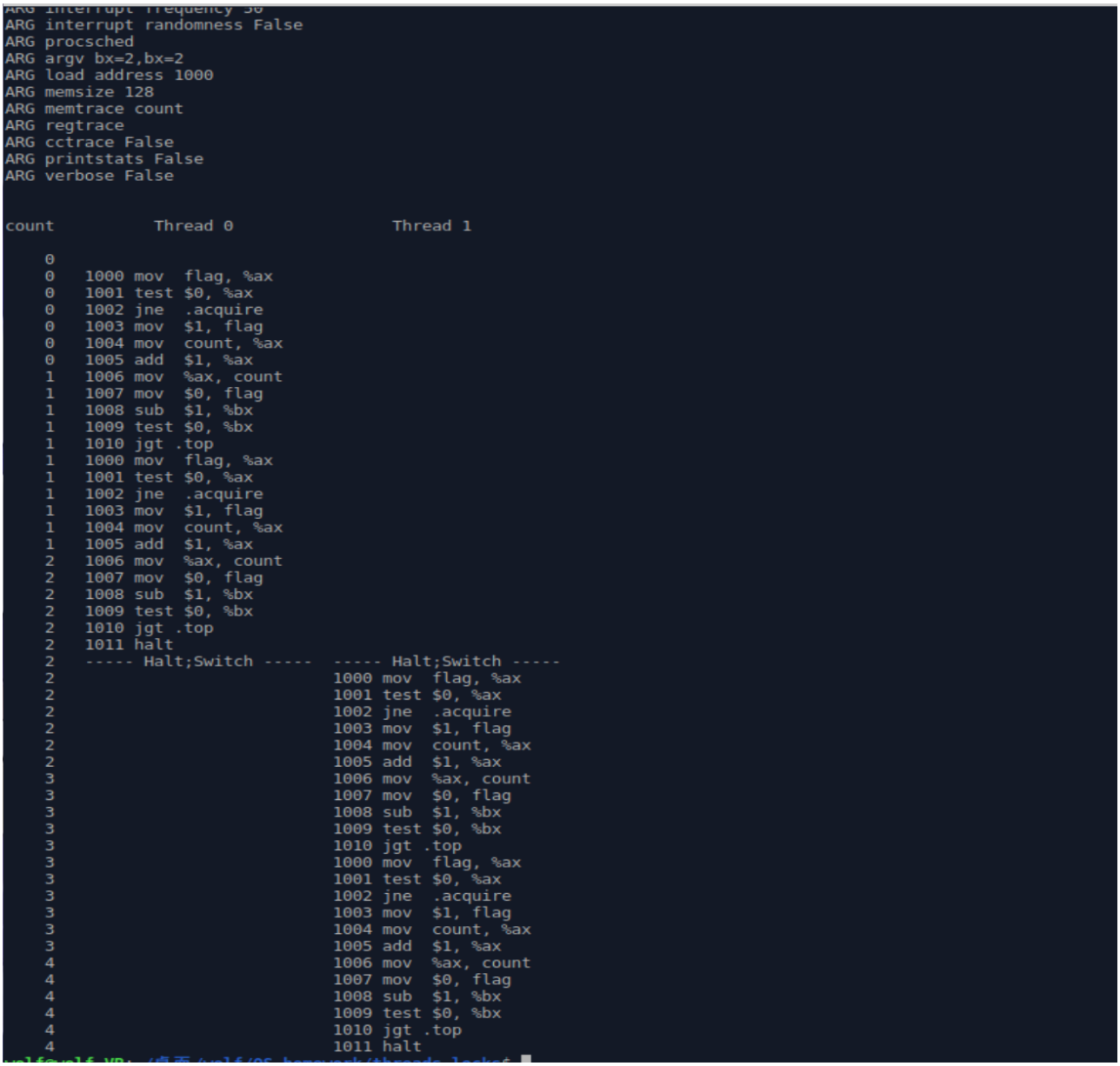
对于count：线程0对count执行了两次加1操作，线程1也对count执行了两次加1操作，所以结束时count的值为4。

对于flag：结束是flag为0

执行如下指令

```
./x86 -p flag.s -t 2 -a bx=2,bx=2 -M count -c
```

运行截图如下：




结果符合预期。

## 28.4

> 对每个线程将 *bx* 设置为高值,然后使用*-i* 标志生成不同的中断频率。什么值导致产生不好的结果?什么值导致产生良好的结果?

将bx设置为高值，即线程会执行很多次循环，指令数目就增加了。

更改中断频率，中断时在一些特定的位置发生线程切换，就会产生在题1指出的可能发生的问题：两线程本应实现count=count+2bx，但运行结果将可能为count<count+2bx。

使用不同的bx值和中断频率测试

即执行代码

```
./x86.py -p flag.s -t 2 -a bx=100,bx=100 -M count -c -i ?
./x86.py -p flag.s -t 2 -a bx=1000,bx=1000 -M count -c -i ?
#? 为相应的中断间隔
```

结果如下

| **BX** | 中断间隔 | 结果（**COUNT**） |
|---|---|---|
| 100 | 20 | 179 |
| 100 | 30 | 182 |
| 100 | 40 | 176 |
| 100 | 50 | 185 |
| 100 | 60 | 194 |
| 100 | 70 | 174 |
| 100 | 80 | 178 |
| 100 | 90 | 182 |
| 100 | 100 | 181 |
| 1000 | 20 | 1779 |
| 1000 | 30 | 1802 |
| 1000 | 40 | 1776 |
| 1000 | 50 | 1985 |
| 1000 | 60 | 1904 |
| 1000 | 70 | 1746 |
| 1000 | 80 | 1800 |
| 1000 | 90 | 1802 |
| 1000 | 100 | 1810 |

按照产生错误的情况来看，应该是中断间隔越小，越多次线程切换，越可能产生错误。但实际上有一些中断间隔下，中断可能总是在设置标志flag的位置发生，就会产生不好的结果，有的中断间隔下，中断总是不发生在设置flag处，结果可能就较好。因此中断间隔越小，结果越不好的这种趋势并不明显。

从总体来说，中断频率越高（i越小），bx越大，越容易产生不好的结果。

## 28.5

> 现在让我们看看程序 *test-and-set.s*。首先尝试理解使用 *xchg* 指令构建简单锁原语的代码。获取锁怎么写?释放锁如何写?

查看汇编文件结果如下：

```
.var mutex
.var count

.main
.top

.acquire
mov  $1, %ax
xchg %ax, mutex     # atomic swap of 1 and mutex
test $0, %ax        # if we get 0 back: lock is free!
jne  .acquire       # if not, try again

# critical section
mov  count, %ax     # get the value at the address
add  $1, %ax        # increment it
mov  %ax, count     # store it back

# release lock
mov  $0, mutex

# see if we're still looping
sub  $1, %bx
test $0, %bx
jgt .top

halt
```

注意到xchg是一个原子操作。其作用为交换ax寄存器与内存mutex空间的值(mutex设为1)。

在之前的flag中，我们取flag并交换%ax与flag的值是分开来进行的，也正是因为如此，中断才会导致问题。

```
mov  flag, %ax      # get flag
test $0, %ax        # if we get 0 back: lock is free!
jne  .acquire       # if not, try again
mov  $1, flag       # store 1 into flag
```

而这里，采取的是原子性的操作完成交换，避免了这个问题。

```
mov  $1, %ax
xchg %ax, mutex     # atomic swap of 1 and mutex
test $0, %ax        # if we get 0 back: lock is free!
jne  .acquire       # if not, try again
```

回答题目的为题如下：

获取锁

```
mov  $1, %ax
xchg %ax, mutex     # 原子操作:交换ax寄存器与内存mutex空间的值(mutex设为1)
test $0, %ax        #
jne  .acquire       # 如果(%ax)!=0则自旋等待,即原mutex值不为0
```

释放锁

```
mov  $0, mutex
```

## 28.6

> 现在运行代码,再次更改中断间隔*(-i)*的值,并确保循环多次。代码是否总能按预期工作*?*有时会导致 *CPU* 使用率不高吗*?*如何量化呢*?*

与第四题相类似，执行如下代码

```
./x86.py -p test-and-set.s -t 2 -a bx=100,bx=100 -M count -c -i ?
./x86.py -p test-and-set.s -t 2 -a bx=1000,bx=1000 -M count -c -i
?
#? 为相应的中断间隔
```

结果如下

---

| BX | 中断间隔 | 结果（COUNT） |
| --- | --- | --- |
| 100 | 20 | 200 |
| 100 | 30 | 200 |
| 100 | 40 | 200 |
| 100 | 50 | 200 |
| 100 | 60 | 200 |
| 100 | 70 | 200 |
| 100 | 80 | 200 |
| 100 | 90 | 200 |
| 100 | 100 | 200 |
| 1000 | 20 | 2000 |
| 1000 | 30 | 2000 |
| 1000 | 40 | 2000 |
| 1000 | 50 | 2000 |
| 1000 | 60 | 2000 |
| 1000 | 70 | 2000 |
| 1000 | 80 | 2000 |
| 1000 | 90 | 2000 |
| 1000 | 100 | 2000 |

可见，代码总能按照预期工作。

这是由于xchg实现的锁能够提供互斥，两线程不会同时进入临界区。

有时CPU利用率不高。主要原因是自旋等待锁被释放的过程也占用了CPU。自旋等待的时间就是CPU利用率不高的原因。

量化计算：完成一次完整的循环计算只需要6条指令，包含获取释放锁在内则需要11条指令，取比值，只有55%的时间CPU用于完成关键的运算。考虑到获取锁还需要等待，完成一次循环计算还需要更多条指令，将预期一次就可以获取锁完成一次计算的11条指令与实际完成一次计算的指令取比值，也可以量化CPU的利用率。

## 28.7

> 使用-*P* 标志生成锁相关代码的特定测试。例如,执行一个测试计划,在第一个线程中获取锁,但随后尝试在第二个线程中获取锁。正确的事情发生了吗?你还应该测试什么?

执行如下代码：

```
./x86.py -p test-and-set.s -i 10 -R ax,bx -M mutex,count -a bx=5 -P
000011111111 -c
```

执行结果如下

```
ARG seed 0
ARG numthreads 2
ARG program test-and-set.s
ARG interrupt frequency 50
ARG interrupt randomness False
ARG procsched 000011111111
ARG argv bx=3
ARG load address 1000
ARG memsize 128
ARG memtrace mutex,count
ARG regtrace ax,bx
ARG cctrace False
ARG printstats False
ARG verbose False


mutex count      ax    bx          Thread 0                Thread 1


    0     0       0     3
    0     0       1     3   1000 mov  $1, %ax
    1     0       0     3   1001 xchg %ax, mutex
    1     0       0     3   1002 test $0, %ax
    1     0       0     3   1003 jne  .acquire
    1     0       0     3   ------ Interrupt ------  ------
Interrupt ------
    1     0       1     3                           1000 mov  $1,
%ax
```

```
    1     0       1     3                          1001 xchg %ax,
mutex
    1     0       1     3                          1002 test $0,
%ax
    1     0       1     3                          1003 jne
.acquire
    1     0       1     3                          1000 mov  $1,
%ax
    1     0       1     3                          1001 xchg %ax,
mutex
    1     0       1     3                          1002 test $0,
%ax
    1     0       1     3                          1003 jne
.acquire
    1     0       0     3   ------ Interrupt ------  ------
Interrupt ------
    1     0       0     3   1004 mov  count, %ax
    1     0       1     3   1005 add  $1, %ax
    1     1       1     3   1006 mov  %ax, count
    0     1       1     3   1007 mov  $0, mutex
    0     1       1     3   ------ Interrupt ------  ------
Interrupt ------
    0     1       1     3                          1000 mov  $1,
%ax
    1     1       0     3                          1001 xchg %ax,
mutex
    1     1       0     3                          1002 test $0,
%ax
    1     1       0     3                          1003 jne
.acquire
    1     1       1     3                          1004 mov
count, %ax
    1     1       2     3                          1005 add  $1,
%ax
    1     2       2     3                          1006 mov  %ax,
count
    0     2       2     3                          1007 mov  $0,
mutex
    0     2       1     3   ------ Interrupt ------  ------
Interrupt ------
    0     2       1     2   1008 sub  $1, %bx
    0     2       1     2   1009 test $0, %bx
```

```
    0     2       1     2   1010 jgt .top
    0     2       1     2   1000 mov  $1, %ax
    0     2       2     3   ------ Interrupt ------  ------
Interrupt ------
    0     2       2     2                           1008 sub  $1,
%bx
    0     2       2     2                           1009 test $0,
%bx
    0     2       2     2                           1010 jgt .top
    0     2       1     2                           1000 mov  $1,
%ax
    1     2       0     2                           1001 xchg %ax,
mutex
    1     2       0     2                           1002 test $0,
%ax
    1     2       0     2                           1003 jne
.acquire
    1     2       2     2                           1004 mov
count, %ax
    1     2       1     2   ------ Interrupt ------  ------
Interrupt ------
    1     2       1     2   1001 xchg %ax, mutex
    1     2       1     2   1002 test $0, %ax
    1     2       1     2   1003 jne  .acquire
    1     2       1     2   1000 mov  $1, %ax
    1     2       2     2   ------ Interrupt ------  ------
Interrupt ------
    1     2       3     2                           1005 add  $1,
%ax
    1     3       3     2                           1006 mov  %ax,
count
    0     3       3     2                           1007 mov  $0,
mutex
    0     3       3     1                           1008 sub  $1,
%bx
    0     3       3     1                           1009 test $0,
%bx
    0     3       3     1                           1010 jgt .top
    0     3       1     1                           1000 mov  $1,
%ax
    1     3       0     1                           1001 xchg %ax,
mutex
```

```
    1     3       1     2   ------ Interrupt ------  ------
Interrupt ------
    1     3       1     2   1001 xchg %ax, mutex
    1     3       1     2   1002 test $0, %ax
    1     3       1     2   1003 jne  .acquire
    1     3       1     2   1000 mov  $1, %ax
    1     3       0     1   ------ Interrupt ------  ------
Interrupt ------
    1     3       0     1                           1002 test $0,
%ax
    1     3       0     1                           1003 jne
.acquire
    1     3       3     1                           1004 mov
count, %ax
    1     3       4     1                           1005 add  $1,
%ax
    1     4       4     1                           1006 mov  %ax,
count
    0     4       4     1                           1007 mov  $0,
mutex
    0     4       4     0                           1008 sub  $1,
%bx
    0     4       4     0                           1009 test $0,
%bx
    0     4       1     2   ------ Interrupt ------  ------
Interrupt ------
    1     4       0     2   1001 xchg %ax, mutex
    1     4       0     2   1002 test $0, %ax
    1     4       0     2   1003 jne  .acquire
    1     4       4     2   1004 mov  count, %ax
    1     4       4     0   ------ Interrupt ------  ------
Interrupt ------
    1     4       4     0                           1010 jgt .top
    1     4       4     0                           1011 halt
    1     4       4     2   ----- Halt;Switch -----  -----
Halt;Switch -----
    1     4       5     2   1005 add  $1, %ax
    1     5       5     2   1006 mov  %ax, count
    0     5       5     2   1007 mov  $0, mutex
    0     5       5     1   1008 sub  $1, %bx
    0     5       5     1   1009 test $0, %bx
    0     5       5     1   1010 jgt .top
```

```
0     5       1     1   1000 mov  $1, %ax
1     5       0     1   1001 xchg %ax, mutex
1     5       0     1   1002 test $0, %ax
1     5       0     1   1003 jne  .acquire
1     5       5     1   1004 mov  count, %ax
1     5       6     1   1005 add  $1, %ax
1     6       6     1   1006 mov  %ax, count
0     6       6     1   1007 mov  $0, mutex
0     6       6     0   1008 sub  $1, %bx
0     6       6     0   1009 test $0, %bx
0     6       6     0   1010 jgt .top
0     6       6     0   1011 halt
```

这里的-P标志用来引起调度0和1号线程的执行，有-P标志存在时-i标志应该会失效，也可以将其删去。

我们先让0号线程执行4句，目的是让0号线程获取锁。再让1号线程执行8句，如果1号线程没有得到锁的话，它会再次执行试图获取锁的代码。从结果上来看，1号线程确实将这段代码执行了两次。如果我们采取0000111111111111这样的调度的话，1号线程会将这段代码执行3次，再更多的话也是像这样。

在1号线程的第二段执行过程那里可以看到，当0号线程释放锁之后，T1再次获取锁成功，随后成功进入临界区执行，这说明这个锁在释放之后，其他线程可以获取锁，锁的功能正常。

## 第30章

运行程序OS-homework/threads-cv/main-two-cvs-while.c

这里注意不是python程序了，打开makefile阅读使用make驱动程序执行的方式。

另外，通过README可知一些重要的标识符如下：

- -p x表示指定生产者数量为x
- -c x表示指定消费者数量为x
- -m x表示指定buffer大小为x
- -l x表示生产多少
- -v跟踪运行的过程

## 30.1

> 我们的第一个问题集中在 *main-two-cvs-while.c*（有效的解决方案）上。 首先，研究代码。 你认为你了解当你运行程序时会发生什么吗？

首先研究该有效解决方案的主要函数（通过注释的方式理解）

```
//这是生产者-消费者模型的最终方案
//缓冲区有不止一个位置，对应的fill和get的实现通过fill_ptr和use_ptr确定执行fill或get的生产者和消费者

//生产者生产内容
void do_fill(int value) {
    // ensure empty before usage
    ensure(buffer[fill_ptr] == EMPTY, "error: tried to fill a non-empty buffer");
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % max;
    num_full++;
}

//消费者获取内容
int do_get() {
    int tmp = buffer[use_ptr];
    ensure(tmp != EMPTY, "error: tried to get an empty buffer");
    buffer[use_ptr] = EMPTY;
    use_ptr = (use_ptr + 1) % max;
    num_full--;
    return tmp;
}

//生产者
//每次生产时，先获取锁，根据缓冲区是否为空决定是否需要休眠等待，为空则调用do_fill生产值，使用fill唤醒消费者，并释放锁
void *producer(void *arg) {
    int id = (int) arg;
    // make sure each producer produces unique values
    int base = id * loops;
    int i;
    for (i = 0; i < loops; i++) {    p0;
        Mutex_lock(&m);              p1;
        while (num_full == max) {    p2;
```

```
            Cond_wait(&empty, &m);  p3;
        }
        do_fill(base + i);          p4;
        Cond_signal(&fill);         p5;
        Mutex_unlock(&m);           p6;
    }
    return NULL;
}

//消费者
//消费者会一直消费至到消费到END_OF_STREAM，消费过程为上锁，根据缓冲区是否有数据，并决定是否休眠，有数据则获取数据，唤醒消费者并释放锁
void *consumer(void *arg) {
    int id = (int) arg;
    int tmp = 0;
    int consumed_count = 0;
    while (tmp != END_OF_STREAM) { c0;
        Mutex_lock(&m);             c1;
        while (num_full == 0) {     c2;
            Cond_wait(&fill, &m);   c3;
        }
        tmp = do_get();             c4;
        Cond_signal(&empty);        c5;
        Mutex_unlock(&m);           c6;
        consumed_count++;
    }
    // return consumer_count-1 because END_OF_STREAM does not count
    return (void *) (long long) (consumed_count - 1);
}
```

可以发现，这段代码实现与书上的P259页的最终的生产者/消费者方案差不多。

代码实现的效果是让生产者生产数据到缓冲区中（如果缓冲区未满），消费者从缓冲区中取数据。程序运行时，希望生产者会在仓库满时等待，并且一旦仓库有空间，则立即解除等待放置数据。消费者会在仓库为空的时候等待，并且仓库一旦有数据，则立刻解除等待消费数据。

## 30.2

> 指定一个生产者和一个消费者运行，并让生产者产生一些元素。 缓冲区大小从 *1* 开始，然后增加。随着缓冲区大小增加，程序运行结果如何改变？ 当使用不同的缓冲区大小*(*例如 *-m 10)*，生产者生产不同的产品数量*(*例如 *-l 100)*， 修改消费者的睡眠字符串*(*例如 *-C 0,0,0,0,0,0,1)*，*full_num* 的值如何变化？

先使用如下指令生成可执行文件

```
make main-two-cvs-while
```

再执行类似如下指令获取结果

```
./main-two-cvs-while -l 100 -m 10 -p 1 -c 1 -v -C 0,0,0,0,0,0,1
```

在终端的运行结果中，NF表示num_full,缓冲区中数据的数据量。中间是仓库的情况。- - -表示该仓库位没有数据。后面P0列表示生成者（producer）0执行到哪一行代码。对应第一题中每一行代码后面的注释，C0列也同理，表示消费者（consumer）0执行到哪一行代码。

①只改变缓冲区

分别执行如下三个指令

```
./main-two-cvs-while -l 3 -m 1 -p 1 -c 1 -v
./main-two-cvs-while -l 3 -m 2 -p 1 -c 1 -v
./main-two-cvs-while -l 3 -m 3 -p 1 -c 1 -v
```

运行截图：

buffer=1

```
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-cv$ ./main-two-cvs-while -l 3 -m 1 -p 1 -c 1 -v
 NF          P0 C0
  0 [*--- ]     c0
  0 [*--- ] p0
  0 [*--- ]     c1
  0 [*--- ]     c2
  0 [*--- ] p1
  1 [*   0 ] p4
  1 [*   0 ] p5
  1 [*   0 ] p6
  1 [*   0 ]     c3
  0 [*--- ]     c4
  0 [*--- ] p0
  0 [*--- ]     c5
  0 [*--- ]     c6
  0 [*--- ] p1
  0 [*--- ]     c0
  1 [*   1 ] p4
  1 [*   1 ] p5
  1 [*   1 ] p6
  1 [*   1 ]     c1
  0 [*--- ]     c4
  0 [*--- ] p0
  0 [*--- ]     c5
  0 [*--- ]     c6
  0 [*--- ] p1
  0 [*--- ]     c0
  1 [*   2 ] p4
  1 [*   2 ] p5
  1 [*   2 ] p6
  1 [*   2 ]     c1
  0 [*--- ]     c4
  0 [*--- ]     c5
  0 [*--- ]     c6
  1 [*EOS ] [main: added end-of-stream marker]
  1 [*EOS ]     c0
  1 [*EOS ]     c1
  0 [*--- ]     c4
  0 [*--- ]     c5
  0 [*--- ]     c6

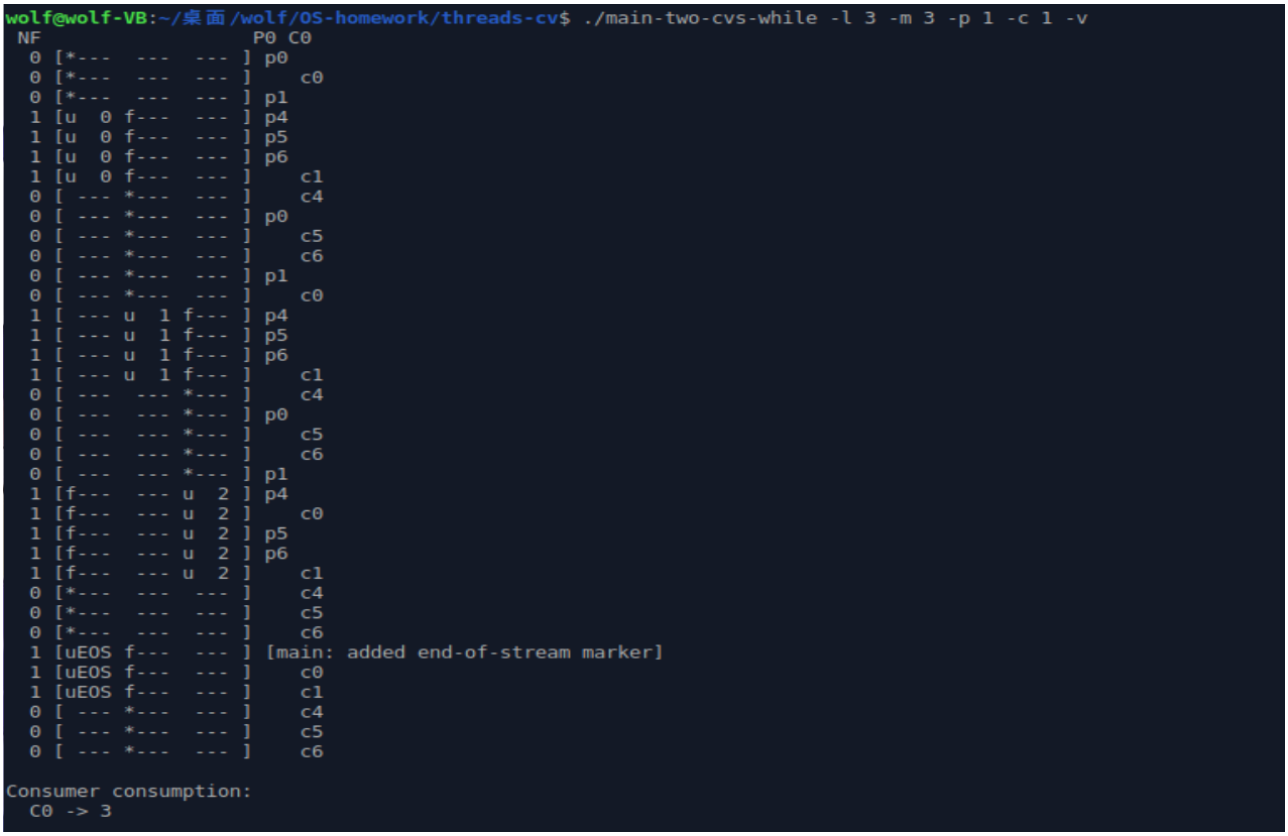
Consumer consumption:
  C0 -> 3
```

buffer=2

```
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-cv$ ./main-two-cvs-while -l 3 -m 2 -p 1 -c 1 -v
 NF             P0 C0
  0 [*---   --- ]     c0
  0 [*---   --- ] p0
  0 [*---   --- ]     c1
  0 [*---   --- ]     c2
  0 [*---   --- ] p1
  1 [u  0 f--- ] p4
  1 [u  0 f--- ] p5
  1 [u  0 f--- ] p6
  1 [u  0 f--- ]     c3
  0 [ --- *--- ]     c4
  0 [ --- *--- ] p0
  0 [ --- *--- ]     c5
  0 [ --- *--- ]     c6
  0 [ --- *--- ] p1
  0 [ --- *--- ]     c0
  1 [f--- u  1 ] p4
  1 [f--- u  1 ] p5
  1 [f--- u  1 ] p6
  1 [f--- u  1 ]     c1
  0 [*---   --- ]     c4
  0 [*---   --- ] p0
  0 [*---   --- ]     c5
  0 [*---   --- ]     c6
  0 [*---   --- ] p1
  0 [*---   --- ]     c0
  1 [u  2 f--- ] p4
  1 [u  2 f--- ] p5
  1 [u  2 f--- ] p6
  1 [u  2 f--- ]     c1
  0 [ --- *--- ]     c4
  0 [ --- *--- ]     c5
  0 [ --- *--- ]     c6
  1 [f--- uEOS ] [main: added end-of-stream marker]
  1 [f--- uEOS ]     c0
  1 [f--- uEOS ]     c1
  0 [*---   --- ]     c4
  0 [*---   --- ]     c5
  0 [*---   --- ]     c6

Consumer consumption:
  C0 -> 3
```

buffer=3




可以看到，程序的运行结果基本不变，m的增加，只是让生成者和消费者每次放置数据的位置和消费数据的有所改变。但消费者还是能够在仓库为空时等待，一旦不为空则开始消费。生成者放置数据没有异常，消费者消费数据也没有异常。最终消费者成功地消费了3个数据。

②设置睡眠序列并固定缓冲区大小

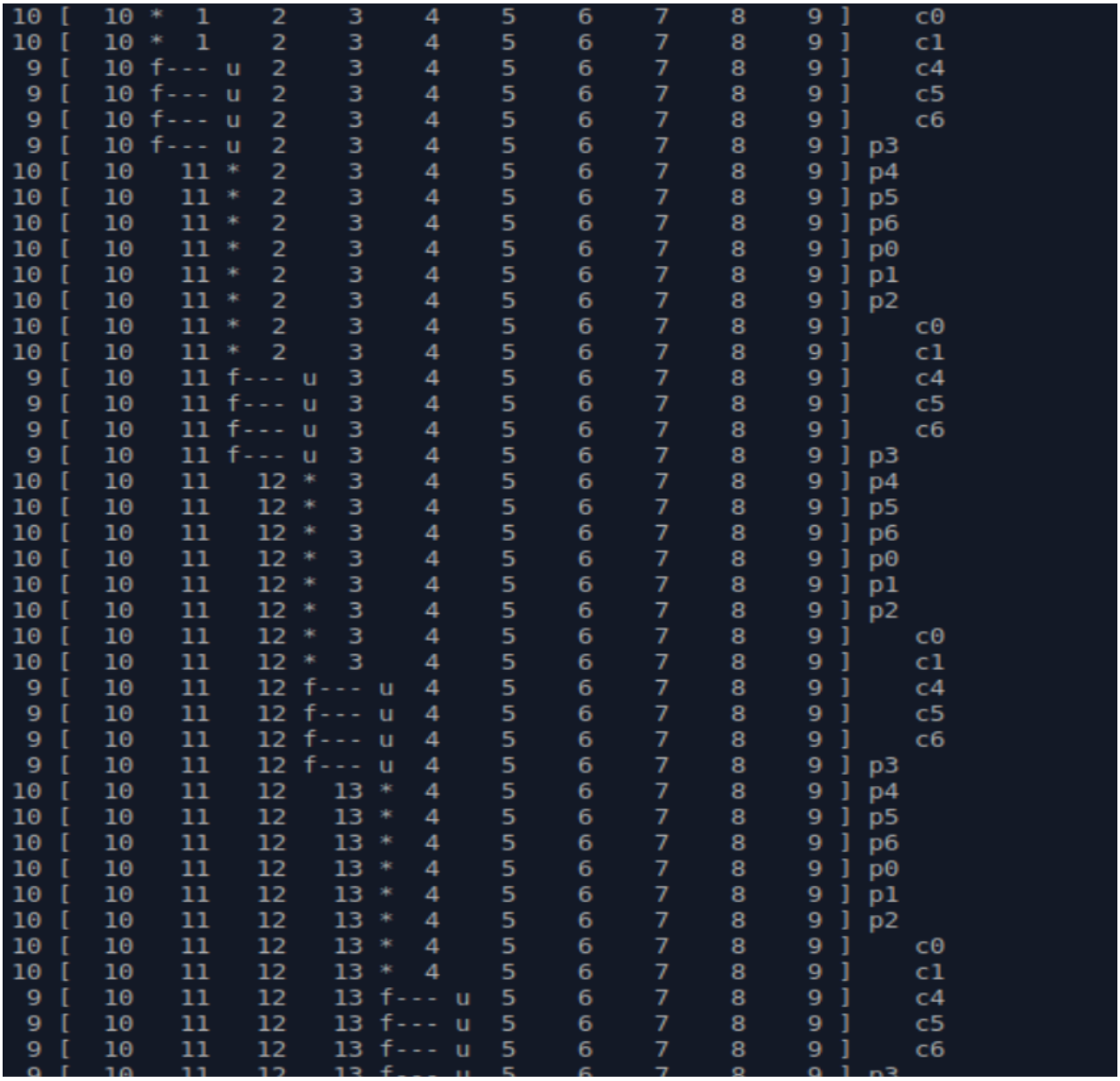
将buffer的大小设置为10(即-m 10)，生产的总数设置为100(即-l 100)，同时设置消费者的睡眠情况为-C 0,0,0,0,0,0,1(也就是在c6处进入睡眠)时，输入如下指令

```
./main-two-cvs-while -p 1 -c 1 -m 10 -l 100 -C 0,0,0,0,0,0,1 -v
```

运行结果部分截图如下：




这是一开始生产者将仓库填满的过程。在此之后会进入如下的交替模式。一旦出现仓库的孔空缺，消费者就会唤醒生产者将空缺填满。




剩下将会按照这个模式交替执行直至100个全部生产完毕后，消费者一次性全部消费完。

## 30.4

> 我们来看一些 *timings*。 对于一个生产者，三个消费者，大小为 *1* 的共享缓冲区以及每个消费者在 *c3* 点暂停一秒，您认需要执行多长时间？ （*./main-two-cvs-while -p 1 -c 3 -m 1 -C 0,0,0,1,0,0,0:0,0,0,1,0,0,0:0,0,0,1,0,0,0 -l 10 -v -t*）

分析：

每次消费者经过c3都要休眠1s，这里因为只有一个缓冲区，所以必然是生产者先生产，然后唤醒消费者消费，故正常情况下应该是10次唤醒，如果不考虑多个消费者同时醒着时因为num_full=0而在while循环里额外的一次，正常应该要10s，再考虑到最终处理EOF标记时3个消费者应该各要1s。故理想情况应该是13s。

运行如下代码：

```
./main-two-cvs-while -p 1 -c 3 -m 1 -C
0,0,0,1,0,0,0:0,0,0,1,0,0,0:0,0,0,1,0,0,0 -l 10 -v -t
```

（睡眠队列-C表示对消费者生效，0001000即在c3时刻睡眠1s）

运行结果：

```
  NF         P0 C0 C1 C2
  0 [*--- ]    c0
  0 [*--- ]       c0
  0 [*--- ]          c0
  0 [*--- ] p0
  0 [*--- ]       c1
  0 [*--- ]       c2
  0 [*--- ]    c1
  0 [*--- ]    c2
  0 [*--- ]          c1
  0 [*--- ]          c2
  0 [*--- ] p1
  1 [*  0 ] p4
  1 [*  0 ] p5
  1 [*  0 ] p6
  1 [*  0 ]       c3
  1 [*  0 ] p0
  0 [*--- ]       c4
  0 [*--- ]       c5
  0 [*--- ]       c6
  0 [*--- ] p1
  0 [*--- ]       c0
  1 [*  1 ] p4
  1 [*  1 ] p5
  1 [*  1 ] p6
  1 [*  1 ]       c1
  0 [*--- ]       c4
  0 [*--- ] p0
  0 [*--- ]       c5
  0 [*--- ]       c6
  0 [*--- ]    c3
  0 [*--- ]       c0
  0 [*--- ]    c2
  0 [*--- ] p1
```

```
  1 [*  2 ] p4
  1 [*  2 ] p5
  1 [*  2 ] p6
  1 [*  2 ]       c1
  0 [*--- ]       c4
  0 [*--- ] p0
  0 [*--- ]       c5
  0 [*--- ]       c6
  0 [*--- ]          c3
  0 [*--- ]       c0
  0 [*--- ]          c2
  0 [*--- ] p1
  1 [*  3 ] p4
  1 [*  3 ] p5
  1 [*  3 ] p6
  1 [*  3 ]       c1
  0 [*--- ]       c4
  0 [*--- ] p0
  0 [*--- ]       c5
  0 [*--- ]       c6
  0 [*--- ]    c3
  0 [*--- ]       c0
  0 [*--- ]    c2
  0 [*--- ] p1
  1 [*  4 ] p4
  1 [*  4 ] p5
  1 [*  4 ] p6
  1 [*  4 ]       c1
  0 [*--- ]       c4
  0 [*--- ] p0
  0 [*--- ]       c5
  0 [*--- ]       c6
  0 [*--- ]          c3
  0 [*--- ]       c0
  0 [*--- ]          c2
  0 [*--- ] p1
  1 [*  5 ] p4
  1 [*  5 ] p5
  1 [*  5 ] p6
  1 [*  5 ]       c1
  0 [*--- ]       c4
  0 [*--- ] p0
```

```
  0 [*---  ]       c5
  0 [*---  ]       c6
  0 [*---  ]    c3
  0 [*---  ]       c0
  0 [*---  ]    c2
  0 [*---  ] p1
  1 [*  6  ] p4
  1 [*  6  ] p5
  1 [*  6  ] p6
  1 [*  6  ]       c1
  0 [*---  ]       c4
  0 [*---  ] p0
  0 [*---  ]       c5
  0 [*---  ]       c6
  0 [*---  ]          c3
  0 [*---  ]       c0
  0 [*---  ]          c2
  0 [*---  ] p1
  1 [*  7  ] p4
  1 [*  7  ] p5
  1 [*  7  ] p6
  1 [*  7  ]       c1
  0 [*---  ]       c4
  0 [*---  ] p0
  0 [*---  ]       c5
  0 [*---  ]       c6
  0 [*---  ]    c3
  0 [*---  ]       c0
  0 [*---  ]    c2
  0 [*---  ] p1
  1 [*  8  ] p4
  1 [*  8  ] p5
  1 [*  8  ] p6
  1 [*  8  ]       c1
  0 [*---  ]       c4
  0 [*---  ] p0
  0 [*---  ]       c5
  0 [*---  ]       c6
  0 [*---  ]          c3
  0 [*---  ]       c0
  0 [*---  ]          c2
  0 [*---  ] p1
```

```
  1 [*  9 ] p4
  1 [*  9 ] p5
  1 [*  9 ] p6
  1 [*  9 ]       c1
  0 [*--- ]       c4
  0 [*--- ]       c5
  0 [*--- ]       c6
  0 [*--- ]    c3
  0 [*--- ]       c0
  0 [*--- ]    c2
  1 [*EOS ] [main: added end-of-stream marker]
  1 [*EOS ]          c3
  0 [*--- ]          c4
  0 [*--- ]          c5
  0 [*--- ]          c6
  0 [*--- ]       c1
  0 [*--- ]       c2
  1 [*EOS ] [main: added end-of-stream marker]
  1 [*EOS ]    c3
  0 [*--- ]    c4
  0 [*--- ]    c5
  1 [*EOS ] [main: added end-of-stream marker]
  1 [*EOS ]    c6
  1 [*EOS ]       c3
  0 [*--- ]       c4
  0 [*--- ]       c5
  0 [*--- ]       c6

Consumer consumption:
  C0 -> 0
  C1 -> 10
  C2 -> 0

Total time: 13.01 seconds
```

可见其他部分花费一点时间，符合预期。但这里可以看出生产者的唤醒具有偏向性，c1消费者消费了所有的产品。

## 30.8

> 现在让我们看一下 *main-one-cv-while.c*。您是否可以假设只有一个生产者， 一个消费者和一个大小为 *1* 的缓冲区，配置一个睡眠字符串，让代码运行出现问题

先看代码

```
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
//生产者
void *producer(void *arg) {
    int id = (int) arg;
    // make sure each producer produces unique values
    int base = id * loops;
    int i;
    for (i = 0; i < loops; i++) {   p0;
        Mutex_lock(&m);             p1;
        while (num_full == max) {   p2;
            Cond_wait(&cv, &m);     p3;
        }
        do_fill(base + i);          p4;
        Cond_signal(&cv);           p5;
        Mutex_unlock(&m);           p6;
    }
    return NULL;
}
                                                                //消费
者
void *consumer(void *arg) {
    int id = (int) arg;
    int tmp = 0;
    int consumed_count = 0;
    while (tmp != END_OF_STREAM) { c0;
        Mutex_lock(&m);             c1;
        while (num_full == 0) {     c2;
            Cond_wait(&cv, &m);     c3;
        }
        tmp = do_get();             c4;
        Cond_signal(&cv);           c5;
        Mutex_unlock(&m);           c6;
        consumed_count++;
    }
```

这与之前的代码的显著区别是生产者与消费者之间的沟通只用一个信号量来解决。而这最明显的一个问题就是在多个消费者的情况下，消费者消费结束时对生产者的唤醒信号可能被另一个消费者接收，这将导致这个消费者检查之后发现无可消费，进而继续睡觉，生产者也未被唤醒，此时三个都在睡觉。导致程序停滞。

本题只有一个生产者和一个消费者，通信对象只是彼此，不会搞混，所以不会出现问题。

本题的答案是构造不出来。

## 30.9

> 现在将消费者数量更改为两个。 为生产者消费者配置睡眠字符串，从而使代码运行出现问题。

问题之前已经分析过了，就是在多个消费者的情况下，消费者消费结束时对生产者的唤醒信号可能被另一个消费者接收，这将导致这个消费者检查之后发现无可消费，进而继续睡觉，生产者也未被唤醒，此时三个都在睡觉。导致程序停滞。这是书上给出的错误。但这种情况由于这里的程序使用了调度很难实现，我们尝试构造别的错误。

考虑以下错误：在生产者结束所有生产后，其中一个消费者被唤醒，在消费完成后，此时唤醒的对象只剩下另一个消费者了。此时两个消费者都无所事事，都睡眠，程序停滞。

构造如下序列

```
0,0,0,0,0,0,1
```

这里注意，我们换代程序执行了，要先make

```
make main-one-cv-while
```

执行如下代码

```
./main-one-cv-while -p 1 -c 2 -m 1 -P 0,0,0,0,0,0,1 -l 3 -v -t
```

运行结果如下

```
  NF         P0 C0 C1
  0 [*--- ]    c0
  0 [*--- ]       c0
  0 [*--- ] p0
```

```
0 [*--- ]       c1
0 [*--- ]       c2
0 [*--- ]    c1
0 [*--- ]    c2
0 [*--- ] p1
1 [*  0 ] p4
1 [*  0 ] p5
1 [*  0 ] p6
1 [*  0 ]       c3
0 [*--- ]       c4
0 [*--- ]       c5
0 [*--- ]       c6
0 [*--- ]    c3
0 [*--- ]    c2
0 [*--- ]       c0
0 [*--- ]       c1
0 [*--- ]       c2
0 [*--- ] p0
0 [*--- ] p1
1 [*  1 ] p4
1 [*  1 ] p5
1 [*  1 ] p6
1 [*  1 ]    c3
0 [*--- ]    c4
0 [*--- ]    c5
0 [*--- ]    c6
0 [*--- ]       c3
0 [*--- ]    c0
0 [*--- ]       c2
0 [*--- ]    c1
0 [*--- ]    c2
0 [*--- ] p0
0 [*--- ] p1
1 [*  2 ] p4
1 [*  2 ] p5
1 [*  2 ] p6
1 [*  2 ]       c3
0 [*--- ]       c4
0 [*--- ]       c5
0 [*--- ]       c6
0 [*--- ]    c3
0 [*--- ]    c2
```

```
  0 [*--- ]       c0
  0 [*--- ]       c1
  0 [*--- ]       c2
  1 [*EOS ] [main: added end-of-stream marker]
  1 [*EOS ]    c3
  0 [*--- ]    c4
  0 [*--- ]    c5
  0 [*--- ]    c6
  0 [*--- ]       c3
  0 [*--- ]       c2
```

如果是自己执行的话，可以发现每次生产者响应之前都会顿一下，这是因为我们设置的1s睡眠，使它在能被响应前都会睡1s。

终端显示，程序在此戛然而止。这是因为c0唤醒c1，但是c1经过检查后发现没有可以消费的对象，并睡眠，此时场上都在睡觉。

## 30.10

> 现在查看 *main-two-cvs-if.c*。 您是否可以配置一些参数让代码运行出现问题？ 再次考虑只有一个消费者的情况，然后再考虑有一个以上消费者的情况。

先查看代码如下：

```
void *producer(void *arg) {
    int id = (int) arg;
    // make sure each producer produces unique values
    int base = id * loops;
    int i;
    for (i = 0; i < loops; i++) {   p0;
    Mutex_lock(&m);             p1;
    if (num_full == max) {      p2;
        Cond_wait(&empty, &m);  p3;
    }
    do_fill(base + i);          p4;
    Cond_signal(&fill);         p5;
    Mutex_unlock(&m);           p6;
    }
    return NULL;
}
```

```
void *consumer(void *arg) {
    int id = (int) arg;
    int tmp = 0;
    int consumed_count = 0;
    while (tmp != END_OF_STREAM) { c0;
    Mutex_lock(&m);             c1;
    if (num_full == 0) {        c2;
        Cond_wait(&fill, &m);   c3;
        }
    tmp = do_get();             c4;
    Cond_signal(&empty);        c5;
    Mutex_unlock(&m);           c6;
    consumed_count++;
    }

    // return consumer_count-1 because END_OF_STREAM does not count
    return (void *) (long long) (consumed_count - 1);
}
```

这也是书上提到的一个错误情况。是语义的问题，获准进入while循环体内部是一个“可能”语义，表示可能拥有此权限但仍然需要核查，这不是一个必然的保证。（书上有更精确的说法）

当只有一个生产者和一个消费者时，如果消费者判断缓冲区为空，休眠等待，生产者生产后唤醒消费者，消费者继续执行c4进行消费，不会产生问题。

当有一个生产者和两个消费者时就会产生问题：消费者休眠后，生产者生产唤醒了该消费者，如果没来得及唤醒并上锁，另一个消费者就进入消费了该值，这个消费者唤醒后执行c4就会发生错误。

只要消费者c1在c0处，而生产者刚完成生产，就可能产生这种问题，不需要特别设定睡眠。

这是一个新的程序，首先要make

```
make main-two-cvs-if
```

执行如下代码

```
./main-two-cvs-if -m 1 -c 2 -p 1 -l 10 -v
```

一次该错误发生的情况截图如下：




解读如下：

```
NF          P0 C0 C1
0 [*--- ] p0
0 [*--- ]       c0
0 [*--- ]    c0
0 [*--- ]       c1
0 [*--- ]       c2
0 [*--- ]    c1
0 [*--- ]    c2
0 [*--- ] p1
1 [*  0 ] p4
1 [*  0 ] p5
1 [*  0 ] p6
1 [*  0 ]       c3
1 [*  0 ] p0
0 [*--- ]       c4
0 [*--- ]       c5
0 [*--- ]       c6
0 [*--- ] p1
0 [*--- ]       c0              //消费者C1已经在c0处
1 [*  1 ] p4
1 [*  1 ] p5
1 [*  1 ] p6                    //生产结束，唤醒c0，但是消费者C1先运行了
1 [*  1 ]       c1
```

```
  1 [*  1 ] p0
  0 [*--- ]       c4              //C1抢先消费了值
  0 [*--- ]       c5
  0 [*--- ]       c6
  0 [*--- ]    c3                 //C0准备消费，将发生错误
  0 [*--- ]       c0
error: tried to get an empty buffer
```

错误就发生了。

## 30.11

> 最后查看 *main-two-cvs-while-extra-unlock.c*。在向缓冲区添加或取出元素时释放锁时会出现什么问题？ 给定睡眠字符串来引起这类问题的发生？ 会造成什么不好的结果？

首先查看代码

```
void *producer(void *arg) {
    int id = (int) arg;
    // make sure each producer produces unique values
    int base = id * loops;
    int i;
    for (i = 0; i < loops; i++) {   p0;
    Mutex_lock(&m);             p1;
    while (num_full == max) {   p2;
        Cond_wait(&empty, &m);  p3;
    }
    Mutex_unlock(&m);
    do_fill(base + i);          p4;
    Mutex_lock(&m);
    Cond_signal(&fill);         p5;
    Mutex_unlock(&m);           p6;
    }
    return NULL;
}


void *consumer(void *arg) {
    int id = (int) arg;
    int tmp = 0;
```

```
    int consumed_count = 0;
    while (tmp != END_OF_STREAM) { c0;
    Mutex_lock(&m);             c1;
    while (num_full == 0) {     c2;
        Cond_wait(&fill, &m);   c3;
        }
    Mutex_unlock(&m);
    tmp = do_get();             c4;
    Mutex_lock(&m);
    Cond_signal(&empty);        c5;
    Mutex_unlock(&m);           c6;
    consumed_count++;
    }
```

特地将do_get 和 do_fill排除出锁控制的范围，可能导致对于缓冲区的访问出现并发问题。

具体而言，如果两个生产者都在执行do_fill，对同一个缓冲区进行操作，对buf[fill_ptr]赋值value，还没有进行fill_ptr+1，此时另一个生产者调度运行，对同一个buf[fill_ptr]赋值value，就导致对同一个缓冲区两次赋值，第一次的赋值被覆盖掉而没有被消费。

再或者，可能两个消费者同时对同一片区域进行消费，造成无数据可取的问题。这是并发的最基础要解决的问题。

错误不太好构造。

# 第31章

## 31.1

> 第一个问题就是实现和测试 *fork/join* 问题的解决方案，如本文所述。 即使在文本中描述了此解决方案， 重新自己实现一遍也是值得的。 *even Bach would rewrite Vivaldi， allowing one soon-to-be master to learn from an existing one*。 有关详细信息，请参见 *fork-join.c*。 将添加 *sleep(1)* 到 *child* 函数内以确保其正常工作。

实现函数

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include "common_threads.h"
```

```c
#include <semaphore.h>

sem_t s;

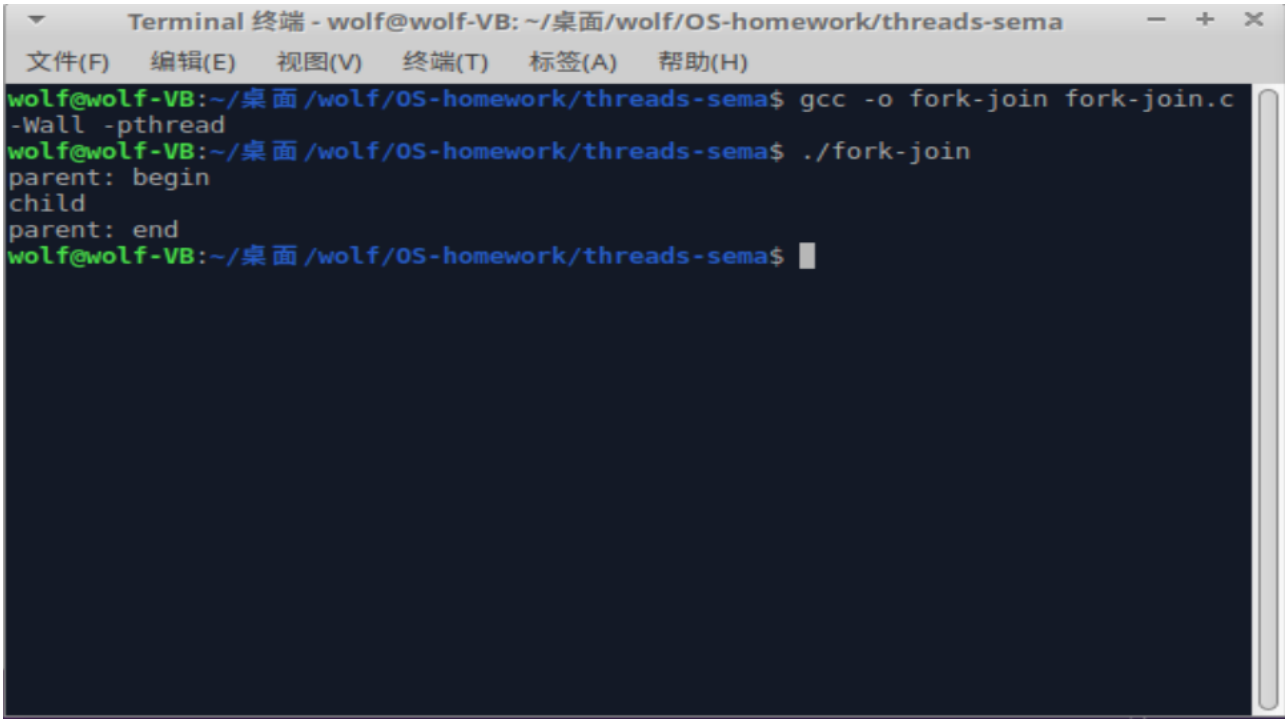
void *child(void *arg) {
    sleep(1);
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p;
    printf("parent: begin\n");
    sem_init(&s, 0, 0);
    Pthread_create(&p, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

执行如下指令编译

```
gcc -o fork-join fork-join.c -Wall -pthread
```

运行结果如下




实际运行时，child出现前有明显停顿1s。

## 31.2

> 现在，我们通过研究集合点问题 *rendezvous problem* 来对此进行概括。 问题如下：您有两个线程，每个线程将要在代码中进入集合点。 任何一方都不应在另一方进入之前退出代码的这一部分。 该任务使用两个信号量，有关详细信息，请参见 *rendezvous.c*。

这个比较简单，主要思路如下：

在每个线程到达集合地点时，发送一个信号量表明自己的状态。每一个线程接收到对方发出的信号量之后才继续往下进行，这就完成了集合。其中任意一个线程的post和wait可以互换顺序，但不能都换。

代码如下：

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include "common_threads.h"
#include <semaphore.h>

// If done correctly, each child should print their "before"
message
```

```
// before either prints their "after" message. Test by adding
sleep(1)
// calls in various locations.

sem_t s1, s2;

void *child_1(void *arg) {
    printf("child 1: before\n");
    sleep(1);
    sem_post(&s1);
    sem_wait(&s2);
    printf("child 1: after\n");
    return NULL;
}

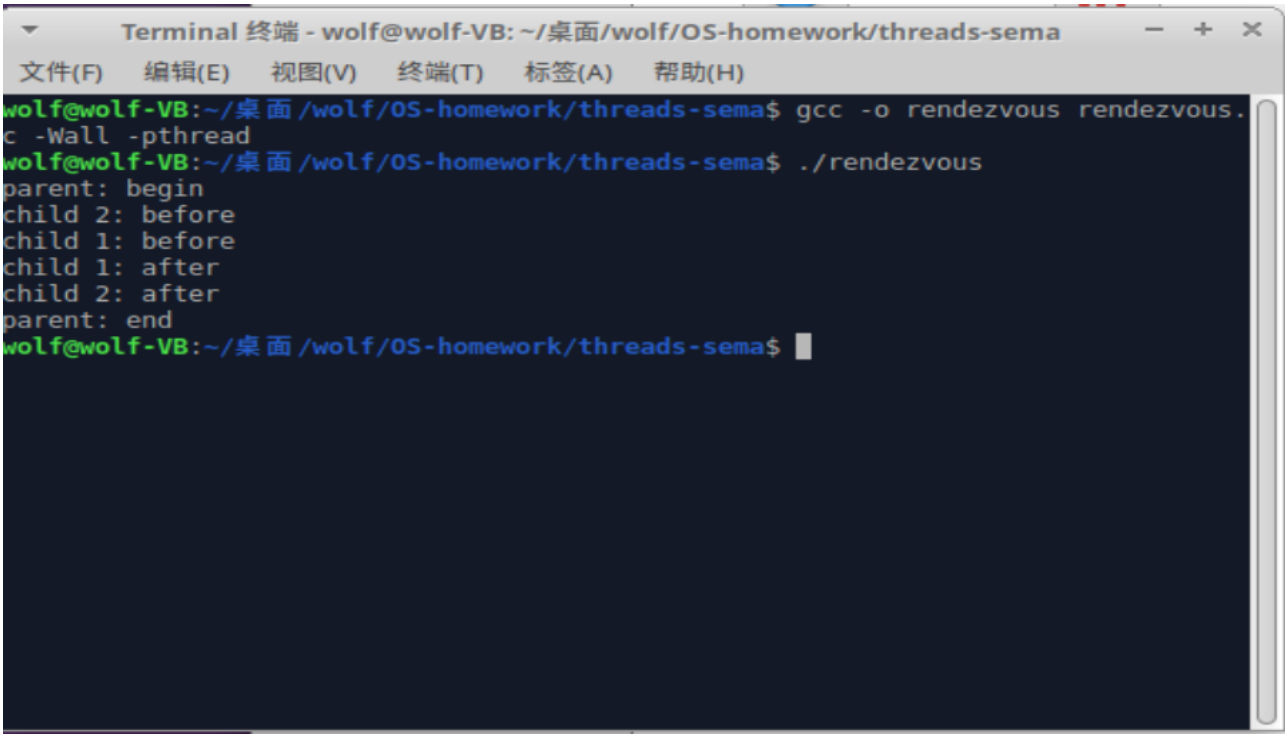
void *child_2(void *arg) {
    printf("child 2: before\n");
    sleep(1);
    sem_post(&s2);
    sem_wait(&s1);
    printf("child 2: after\n");
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    printf("parent: begin\n");
    sem_init(&s1, 0, 0);
    sem_init(&s2, 0, 0);
    Pthread_create(&p1, NULL, child_1, NULL);
    Pthread_create(&p2, NULL, child_2, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("parent: end\n");
    return 0;
}
```

执行如下指令编译

```
gcc -o rendezvous rendezvous.c -Wall -pthread
```

运行截图如下




## 31.4

> 现在按照文本中所述，解决读者写者问题。 首先，不用考虑进程饥饿。 有关详细信息，请参见 *reader-writer.c* 中的代码。 将 *sleep*（）调用添加到您的代码中，以证明它可以按预期工作。 你能证明饥饿问题的存在吗？

这道题还是蛮复杂的，要求实现5个函数和一个结构体定义。

阅读书P272内容学习了书上的代码后，我们可以惊喜地发现，这两者是完全一致的。

代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "common_threads.h"
#include <semaphore.h>
//
// Your code goes in the structure and functions below
//

typedef struct __rwlock_t {
    sem_t lock;
    sem_t write_lock;
```

```
    int reader_number;
} rwlock_t;

//初始化锁
void rwlock_init(rwlock_t *rw) {
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->write_lock, 0, 1);
    rw->reader_number = 0;
}

void rwlock_acquire_readlock(rwlock_t *rw) {
    sleep(1);
    sem_wait(&rw->lock);//获取访问reader_number的锁
    rw->reader_number++;
    if (rw->reader_number == 1) {
        //第一个读者获取写锁，防止该锁被写者获取
        sem_wait(&rw->write_lock);
    }
    sem_post(&rw->lock);//释放访问reader_number的锁
}

void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);//获取访问reader_number的锁
    rw->reader_number--;
    if (rw->reader_number == 0) {
        //最后一个读者释放写锁
        sem_post(&rw->write_lock);
    }
    sem_post(&rw->lock);//释放访问reader_number的锁
}

void rwlock_acquire_writelock(rwlock_t *rw) {
    sleep(1);
    sem_wait(&rw->write_lock);//写者获取写锁
}

void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->write_lock);//写者释放写锁
}

//
// Don't change the code below (just use it!)
```

```
//

int loops;
int value = 0;

rwlock_t lock;

void *reader(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        rwlock_acquire_readlock(&lock);
        printf("read %d\n", value);
        rwlock_release_readlock(&lock);
    }
    return NULL;
}

void *writer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        rwlock_acquire_writelock(&lock);
        value++;
        printf("write %d\n", value);
        rwlock_release_writelock(&lock);
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    assert(argc == 4);
    int num_readers = atoi(argv[1]);
    int num_writers = atoi(argv[2]);
    loops = atoi(argv[3]);
    pthread_t pr[num_readers], pw[num_writers];
    rwlock_init(&lock);
    printf("begin\n");
    int i;
    for (i = 0; i < num_readers; i++)
        Pthread_create(&pr[i], NULL, reader, NULL);
    for (i = 0; i < num_writers; i++)
        Pthread_create(&pw[i], NULL, writer, NULL);
```

```
    for (i = 0; i < num_readers; i++)
        Pthread_join(pr[i], NULL);
    for (i = 0; i < num_writers; i++)
        Pthread_join(pw[i], NULL);

    printf("end: value %d\n", value)
    return 0;
}
```

执行如下指令编译代码

```
gcc -o reader-writer reader-writer.c -Wall -pthread
```

执行如下指令测试代码

```
./reader-writer 5 5 10
```

结果如下：

```
begin
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
```

```
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
read 0
write 1
write 2
write 3
write 4
write 5
write 6
write 7
write 8
write 9
write 10
write 11
write 12
write 13
write 14
write 15
write 16
write 17
write 18
write 19
write 20
write 21
write 22
```

```
write 23
write 24
write 25
write 26
write 27
write 28
write 29
write 30
write 31
write 32
write 33
write 34
write 35
write 36
write 37
write 38
write 39
write 40
write 41
write 42
write 43
write 44
write 45
write 46
write 47
write 48
write 49
write 50
read 50
read 50
read 50
read 50
read 50
read 50
read 50
read 50
read 50
read 50
end: value 50
```

运行很快，几乎无停顿，结果倒也符合预期，只是read和write的分布过于集中。

## 31.5

> 让我们再次看一下读者写者问题，但这一次需要考虑进程饥饿。 您如何确保所有读者和写者运行？ 有关详细信息，请参见 *reader-writer-nostarve.c*。

本题需要解决读者写者锁中写者可能饿死的问题。写者可能饿死，是因为读者数量不受限制，同一时刻可以有多个读者进行读，而只要有读者，写者就不能获取写锁。

为了解决这个问题，可以再增加一个信号量实现一个锁write_waiting_lock，一旦有写者准备进行写操作，尝试获取该锁，获取该锁后可以使新的读者不能进行读，直到写者获取写锁为止。

这样读者的数量就受到了限制，写者能够保证在等待当前数量的读者读取数据后，可以进行写操作，而不需要因为一直有新加入的读者而等待至饿死。

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "common_threads.h"
#include <semaphore.h>
//
// Your code goes in the structure and functions below
//

typedef struct __rwlock_t {
    sem_t lock;
    sem_t write_lock;
    sem_t write_waiting;
    //该锁定义等待锁
    //写者尝试获取写锁时先获取等待锁，禁止新读者加入
    int reader_number;
} rwlock_t;


void rwlock_init(rwlock_t *rw) {
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->write_lock, 0, 1);
    sem_init(&rw->write_waiting, 0, 1);
    rw->reader_number = 0;
}

void rwlock_acquire_readlock(rwlock_t *rw) {
    sleep(1);
```

```
    sem_wait(&rw->write_waiting);//读者读取时必须先获得等待锁以证明此时没有写者存在，也就是自己不是阻碍写者的新变量
    sem_wait(&rw->lock);
    rw->reader_number++;

    if (rw->reader_number == 1) {
        sem_wait(&rw->write_lock);
    }
    sem_post(&rw->lock);
    sem_post(&rw->write_waiting);//读者获取了读锁，归还等待锁

}

void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->reader_number--;
    if (rw->reader_number == 0) {
        sem_post(&rw->write_lock);
    }

    sem_post(&rw->lock);
}

void rwlock_acquire_writelock(rwlock_t *rw) {
    sleep(1);
    sem_wait(&rw->write_waiting);//写者先获得等待锁，禁止新读者加入
    sem_wait(&rw->write_lock);
    sem_post(&rw->write_waiting);//写者已经获取写权限，归还等待锁
}

void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->write_lock);
}

//
// Don't change the code below (just use it!)
//

int loops;
int value = 0;

rwlock_t lock;
```

```c
void *reader(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        rwlock_acquire_readlock(&lock);
        printf("read %d\n", value);
        rwlock_release_readlock(&lock);
    }
    return NULL;
}

void *writer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        rwlock_acquire_writelock(&lock);
        value++;
        printf("write %d\n", value);
        rwlock_release_writelock(&lock);
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    assert(argc == 4);
    int num_readers = atoi(argv[1]);
    int num_writers = atoi(argv[2]);
    loops = atoi(argv[3]);

    pthread_t pr[num_readers], pw[num_writers];

    rwlock_init(&lock);

    printf("begin\n");

    int i;
    for (i = 0; i < num_readers; i++)
        Pthread_create(&pr[i], NULL, reader, NULL);
    for (i = 0; i < num_writers; i++)
        Pthread_create(&pw[i], NULL, writer, NULL);

    for (i = 0; i < num_readers; i++)
        Pthread_join(pr[i], NULL);
```

```
    for (i = 0; i < num_writers; i++)
        Pthread_join(pw[i], NULL);

    printf("end: value %d\n", value);

    return 0;
}
```

执行如下指令编译代码

```
gcc -o reader-writer-nostarve reader-writer-nostarve.c -Wall -
pthread
```

执行如下指令测试代码

```
./reader-writer-nostarve 5 5 10
```

结果如下：

```
begin
read 0
read 0
read 0
read 0
write 1
write 2
write 3
write 4
write 5
read 5
read 5
read 5
read 5
write 6
write 7
read 7
write 8
write 9
write 10
read 10
```

```
read 10
read 10
write 11
write 12
read 12
write 13
read 13
write 14
write 15
read 15
read 15
read 15
write 16
write 17
read 17
write 18
read 18
write 19
write 20
read 20
read 20
read 20
write 21
write 22
read 22
write 23
read 23
write 24
write 25
read 25
write 26
read 26
write 27
write 28
read 28
read 28
write 29
read 29
write 30
read 30
write 31
read 31
```

```
write 32
read 32
read 32
write 33
write 34
read 34
write 35
read 35
read 35
write 36
read 36
read 36
write 37
write 38
write 39
read 39
write 40
read 40
read 40
write 41
read 41
write 42
write 43
write 44
read 44
write 45
read 45
read 45
write 46
read 46
write 47
write 48
write 49
read 49
write 50
read 50
read 50
read 50
end: value 50
```

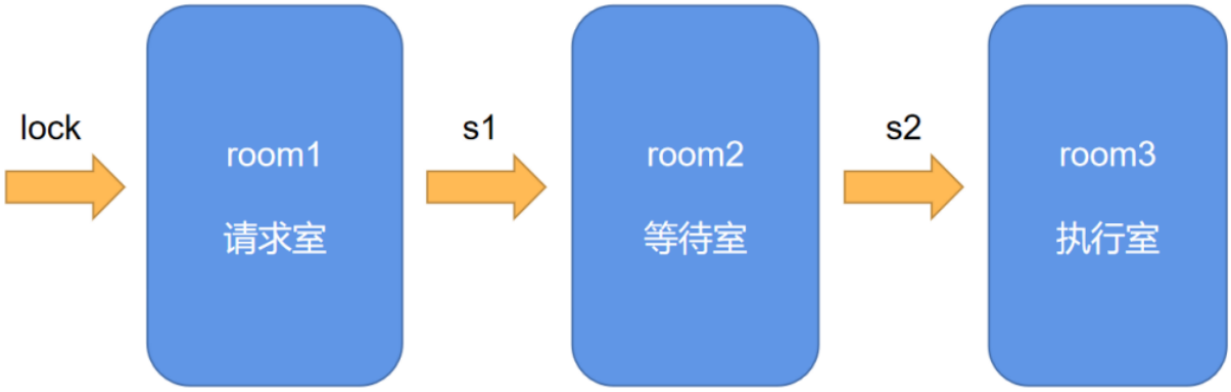
相比刚刚已经有了明显的改进。至少读写的分布已经均衡很多了，解决了饿死的问题。

## 31.6

> 使用信号量构建一个没有饥饿的互斥量，其中任何试图获取该互斥量的线程都将最终获得它。 有关更多信息，请参见 *mutex-nostarve.c* 中的代码。

该题参考了LittleBookOfSemaphores中的Morris's Solution

我们设置有三个线程waiting空间：room1，room2，room3




其中room3是隐式设置的，里面同一时间只能有一个线程，也可以说是互斥的

原理如下：

- 先将一段时间内发出获取锁请求的线程都放入room1
- 由于信号量s1初始值为1，第一个进入room1的线程可以继续执行48行之后的内容，它可以进入room2。
- 它在room2中进行了一个操作，就是判断自己之后是否还有线程在room1中被阻塞，若有的话，随机唤醒一个进入room2，若没有了，就通知room2中的线程可以依次进入room3。
- 当room1中的线程全部进入room2，最后一个线程发出s2的post，room2中的随机一个线程被唤醒，，离开room2（进入room3），执行线程的主体程序，然后释放锁。
- 在释放锁时，线程会回头判断身后room2中是否还有线程被阻塞，若有，随机唤醒一个进入room3，若没有了，就通知这段时间内被阻塞在room1门外也就是图中代码第42行的线程可以依次进入room1。

我可以做一个比喻：

- room1是教室
- room2是面试等待室
- room3是面试场
- 同学依次进入教室，每次只能进一个人，先握住门把手，把自己的名字写在教室的名单上，再放开门把手。

- 一批同学在教室里，一起进入面试等待室，此时每个同学需要依次同时握住教室的门把手和面试等待室的门把手，并同时先从教室的名单上划去自己的名字，然后同时在面试等待室的名单上写上自己的名字，然后松开两个教室的门把手，进入面试等待室。
- 这个进入面试等待室的同学，如果发现教室内还有同学，他就要把等待室的锁开着，自己在等待室等着；如果教室内没有同学了，自己是最后一个同学了，他就打开面试场的锁。
- 面试场的锁开了，等待室的某一个同学握住面试场的门把手，在等待室的名单上划去自己的名字。然后进入面试场地。
- 同学离开面试场地时，如果等待室有同学，就把面试场的门锁开着，否则自己是最后一个人了，那这一批结束，全部重置，等待下一批同学来到教室。

实现代码如下：

```
#include "common_threads.h"
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

//
// Here, you have to write (almost) ALL the code. Oh no!
// How can you show that a thread does not starve
// when attempting to acquire this mutex you build?
//

typedef struct __ns_mutex_t
{
    int room1; // 请求线程列表，lock信号量为锁
    int room2; // 等待线程列表，s1信号量为锁
    sem_t s1;
    sem_t s2;
    sem_t lock;
} ns_mutex_t;

void ns_mutex_init(ns_mutex_t *m)
{
    sem_init(&m->s1, 0, 1);    // s1初始值为1，允许1个线程进入修改room2
    sem_init(&m->s2, 0, 0);    // s2初始值为0，等待获取room2能进入room3的
信号
    sem_init(&m->lock, 0, 1); // lock初始值为1，一次只允许1个线程进入修改
room1
```

```
    m->room1 = 0;
    m->room2 = 0;
}

void ns_mutex_acquire(ns_mutex_t *m)
{
    // 进入room1
    sem_wait(&m->lock); // 等待进入room1
    m->room1++;         // 进入room1
    sem_post(&m->lock);

    // 等待点1：room1

    // 离开room1，进入room2
    // 这里同时占有两个锁
    sem_wait(&m->s1);
    m->room2++;
    sem_wait(&m->lock);
    m->room1--;

    // 将room1中的所有线程放入room2，并打开room3的锁
    if (m->room1)
    { // 若room1内还有线程等待，开启room2的锁，本线程需要将它放入room2，然后本线程也等待在room2
        sem_post(&m->lock);
        sem_post(&m->s1);
    }
    else
    { // 若room1内无线程等待，本线程是最后一个线程，开启room3的锁，允许room2的进入room3
        sem_post(&m->lock);
        sem_post(&m->s2);
    }

    // 等待点2：room2

    //  离开room2，进入room3（也就是开始执行）
    sem_wait(&m->s2);
    m->room2--;
}

void ns_mutex_release(ns_mutex_t *m)
```

```
{
    // 本线程执行完后，放行一个来自room2的线程
    if (m->room2)
    {
        sem_post(&m->s2); // 打开room3的门
    }
    else
    {
        sem_post(&m->s1); // 恢复初始值，重新开始上述例程
    }
}

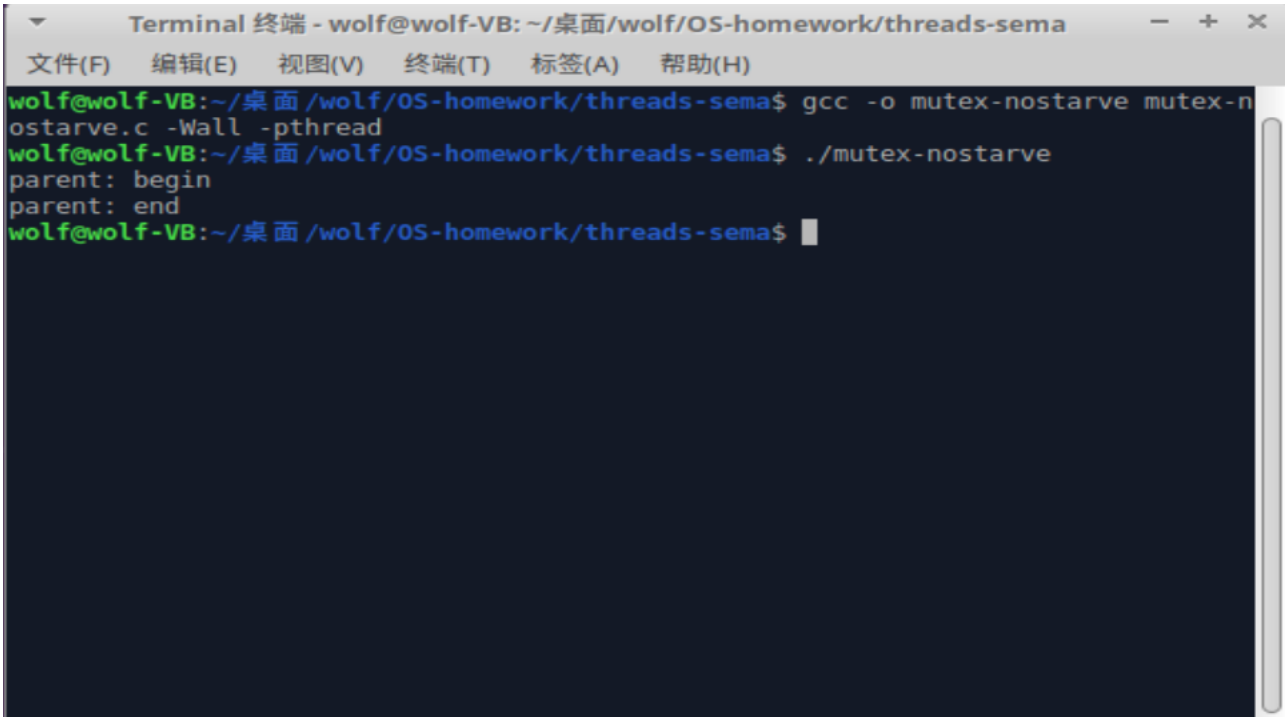
void *worker(void *arg)
{
    return NULL;
}

int main(int argc, char *argv[])
{
    printf("parent: begin\n");
    printf("parent: end\n");
    return 0;
}
```

执行如下命令

```
gcc -o mutex-nostarve mutex-nostarve.c -Wall -pthread
```

运行结果：


Terminal 终端 - wolf@wolf-VB: ~/桌面/wolf/OS-homework/threads-sema
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-sema$ gcc -o mutex-nostarve mutex-nostarve.c -Wall -pthread
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-sema$ ./mutex-nostarve
parent: begin
parent: end
wolf@wolf-VB:~/桌面/wolf/OS-homework/threads-sema$